Project Record: Utensil Theories Transfer Protocol

Increasing an AI's comprehension to improve communication - a full record, 17 to 21 October 2026

Prepared for the project owner, 21 October 2026.

Authorship: Garrett C. Gauthier is the owner and sole author of this project and its ideas. Testing was done with Claude (Anthropic) in Claude Code, a tool the owner paid for.

How to read this:

- Start with section 1, the short version (one page).
- Section 3 is the full timeline, day by day: every request, what was done, and what changed.
- Section 12 explains every technical term in plain words.

Contents

1 The short version
2 The goal, and how it was sharpened
3 Timeline, beginning to end
4 The owner's ideas, in their own words, and what was built from each
5 Every experiment (part A: the first script and experiments 1 to 11; part B: the seed test, the graph method, the greenhouse ladder, the numbers test)
6 Changes, adaptations, mistakes and corrections
7 The comprehension test across AIs
8 What already exists in research, and where the owner's credit could come from
9 Where things stand now, and what comes next
10 Every file and where it lives
11 How to reproduce the work
12 Plain-language glossary
13 Things outside the AI's control

1. The short version

The goal, in the owner's words: "Increase an LLM's ability to 'Comprehend' and 'Understand' in a way that improves communication." The Bible, fables and moral stories were training material, not the goal.

How it was tested. Tiny language models (about 3.4 million numbers each, trained from scratch on the King James Bible on a 4-core cloud computer) were scored on stories and verses they had never seen. Each test had controls. Where possible, the pass/fail rules were written down before any result was seen.

Strong and repeated:

- **Marking right and wrong in the training text helps.** A tiny model then judges new stories better: about 69 right per 100, against about 50 with no marking (experiments 8 and 10). This was a side measure, not the pre-registered main one.
- **Understanding shrinks the exact seed** (experiment 12, confirmed). The tiny Bible model regrew 623 verses it had never seen (83,757 characters) byte for byte from a 12.2 KB seed: 85.4% smaller, and less than half of what xz (a standard compression tool) needs. Each rung of shared understanding gave a smaller seed. The principle is already published (Shannon, 1951 onward). The measurement here is the project's own.

Supported once: the owner's carried balance (the shifting word) beat a plain good/bad label only in long stories, where the act had scrolled out of the model's view. That was about 59 vs 53 right per 100 ($p = 0.004$, meaning luck alone would give this about 4 times in 1,000), in a re-run after two fixes, with about 65 of 100 pairs tied.

Not confirmed:

- predicting consequences (chance every time);
- Aesop morals carrying across cultures (no training set beat chance);
- the shifting word making seeds smaller (they came out about 2% larger);
- the shifting word beating the plain label in short stories (a tie).

The seed and graph method regrows any file exactly.

- It shrinks patterns (99.99%), but not ordinary text (6-19%, where zip manages 70%) or random data (0%).
- The owner's reported 98.9% is most likely explained by a shared library: naming an item already on the shelf. With the Bible on the shelf this measured 98.6%.

Running now: the first test of the owner's own recharge rule, "W1 = 17 + 4 Remaining", with the owner's numbers 17, 32, 16, 34 and 47.

- 18 tiny models are being trained.
- The test passed an independent review and two dry runs.
- The full run started at 17:45 UTC on 21 October and takes about 4 to 5 hours. No results yet.

Queued (owner-approved):

- the "2" test: numbers the model can read;
- then the +/- agent, with the owner's scoring rule: many small negatives, fewer but larger positives, every input pulling a little negative, and the two sides balancing.

The honest limit. Everything here comes from tiny models. None of it changes how a large AI understands anyone. The next real step needs outside help: more computing power, a large-model API key (an access code for a large AI's paid service), or a fresh expert reading.

2. The goal, and how it was sharpened

2.1 The goal, in the owner's words

On 18 October (05:46 UTC) the owner wrote the objective down:

> *"Increase an LLM's ability to "Comprehend" and "Understand" in a way that improves communication."*

An LLM (large language model) is an AI that reads and writes text, such as Claude, Gemini or Copilot. The owner said the same thing in other words along the way:

- "I am trying to teach actual "comprehension" and "understanding" for A.I. not about people" (17 Oct, 21:52).
- "I want to improve AI compression and understanding" (20 Oct, 09:30).

Working definition used in every test: comprehension means applying meaning to a new situation the model has never seen. Repeating memorised patterns (rote) does not count. Every test is therefore scored on new material against a coin toss (50% = chance). Each one also has a control (the same setup with the meaningful part scrambled) and pass/fail rules saved before any result is seen ("pre-registered").

2.2 The first framing, and why it was dropped

The work began with the owner's script, a "Contextual Resonance Engine". It spotted words like guilt, regret and betrayal in a person's message, scored their moral and emotional "weight", and picked a response stance such as "Atlas Protocol". The AI fixed its bugs and added 16 tests (`resonance/`, commit `d8ebeec`; a commit is a saved, dated change in the repository). It also said plainly that the script's "wavelength" was a fingerprint of the text, not a measurement. Keyword matching is shallow, and labelling a user's words "sin" means judging the person.

Eleven minutes after the first message, the owner said the goal was the AI's comprehension, "not about people". From then on the question was this: can a reader that has only words, like a person born blind learning colour, build real understanding? And do rules, plus the human errors that break them, help? The script stays in `resonance/` as a record.

2.3 Moral content was training data, not the goal

The owner's theory was that rules plus human mistakes give meaning its weight, and that lessons told as stories transfer further than lessons told as rules. So the tests used the King James Bible, folk tales and Aesop's fables. By the second day the work had drifted: it chased moral-judging scores and lost sight of communication. Three things pulled it back:

- **18 Oct, 05:46.** The owner asked the AI to "ensure that your drifting does not pass to the next llm". The AI found the objective was not written anywhere up front.
- **The audit (second session, 18 Oct).** "All 11 experiments measure moral judging of Bible-style sentences in a 3.4-million-parameter model. None measures communication, and none uses a real-size AI." It put the project at "about rung 1 of 5".
- **18 Oct, 15:29.** The owner: "The moral aspect was merely training data that you drifted with." The AI rewrote the method without the moral content. The habits it named were: mark what matters, carry it forward, let clear moments reset the state and small details nudge it, and keep a running "state note" with "DECIDED:" and "REJECTED:" markers in a conversation.

2.4 The drift guard (`experiments/HANDOFF.md`)

The AI added an "Objective and drift guard" section to the top of the handoff file on 18 Oct at 05:47 (commit `2b3d28e`). Every new session reads it first. It says:

- **The tests are proxies, not the goal.** Never chase a benchmark number for its own sake.
- **The Bible, the fables and the moral content are training material.**
- **Communication is the payoff:** through `COMMUNICATION_GUIDE.md`, and through the carried-state idea (can a model keep track of what matters across a long exchange?).
- **Scale honesty:** tiny-model results do not change how a large AI like Claude understands anyone.
- **Tone:** calibrated and plain. No "this could be huge" unless a test says so.
- **The first framing (reading people) was dropped.**

The AI named the three risks at the same time: proxy drift, scale drift and tone drift. The audit prompt the owner approved repeats the guard: "If a step does not move us toward better communication, say so before you do it."

2.5 Later sharpening (20-21 October)

- **Understanding as compression ("the greenhouse").** A model that understands text better needs a smaller exact "seed" to regrow it. That makes understanding one measurable number (experiment 12).
- **Comprehension as behaviour.** The owner's Gemini and Copilot test showed comprehension as things you can score: staying with what is given, saying what is unclear, asking before assuming, keeping unrelated memory out. The memory an AI carries in was read as the owner's "+4 remaining".
- **"Nothing is useless."** Every input is "+" or "-" information added to a running total, which is gauged only by its placement on a line. This became the queued "+/-" agent.
- **Still untested:** communication itself. The conversation test (PLAN step 1b) has not been run, and no large AI has been used as the reader. That needs an API key the owner would add.

3. Timeline, beginning to end

How to read this timeline:

- Times are UTC. The owner's local date may differ.
- "#n" is the owner's message number (59 in all).
- Short codes such as `d8ebeec` are commits: saved, dated changes in the repository.

Tags:

- **[DIRECTION]** the request changed direction;
- **[CORRECTION]** the owner corrected the AI;
- **[PAUSE]** work stopped on purpose;

- **[LIMIT]** a usage-limit stop;
- **[RESTART]** a step was re-run or resumed;
- **[BLOCKED]** a safety check stopped the reply.

Earlier commits (`b8057a8`, `75bbff6`, `690d671`, July-August) belong to the owner's separate zine project, which is out of scope.

17 October 2026 (Saturday)

- **21:41 #1** - The owner asked to "Evaluate and enhance" the resonance script. The AI ran it and confirmed the bugs: missed word forms, ignored negation, one word triggered "grace", one stance could never fire, and the "wavelength" changed every run. It rebuilt the script with 16 tests (`d8ebeec`).
- **21:47 #2** - "Is this feasible, am I onto something?" The AI said yes, as a tone guard, but it was half tool and half metaphor.
- **21:50 #3, 21:51 #4** - The owner pasted a chat with another AI about "the architecture of human error" (Commandments, unforgivable sins, "A mistake ... is the calibration point") and asked the AI to separate the idea from the task. The chat held personal reflections, which are not reproduced here. The AI warned of the other AI's "agreement loop" and mapped the idea as rule -> deviation -> weight -> direction -> grace.
- **21:52 #5 [DIRECTION]** - "...for A.I. not about people".
- **22:02 #6 [DIRECTION]** - The blind-colour question; the Bible as "a road map to understanding"; "have faith and give it a try. If it doesn't work we will try something else." Experiments 1-4 on the King James Bible found:
  - colour matched 10 of 14 (chance 17%);
  - rule verses alone were no better than other verses;
  - "covet" weakened by about half without the rules.

Commit `ec22991`.

- **22:12 #7** - "try the consequences test". Experiment 5: chance (45-57%) on 20 new story pairs (`939d60f`).
- **22:19 #8** - "try the tiny transformer". Experiment 6: a tiny modern-design model (a "transformer") learned Bible language in 12 minutes, but scored chance on the stories (45-55%) (`df94814`, `1a4dfac`).
- **23:00 #9 [DIRECTION]** - The owner's shifting word: "[w1=17 -<next word ... (-13)> = +4 Remaining]". It resets to +17 or -17 on a clear right or wrong, its spellings vary like "to, too, two, 2", and it should "shift the coin toss". The owner also noted their numbers 17, 32, 16, 34, 47. The AI built "velanok": five syllable-rotation spellings and a fading balance. It switched off the owner's re-adding of 17 ("17 + 4"), because the balance would grow forever. Experiment 7: 68% vs 52% for the scrambled control, on 20 pairs (`f676eed` to `c918cb7`). This was later shown to be luck, and no rules had been saved first.

18 October 2026 (Sunday)

- **00:46 #10** - "Let's confirm the result". The owner also asked what it would mean for them and whether it could earn anything. The AI saved the rules and 100 new pairs before training (`5538b46`). Its opinion: a small, real finding if confirmed; its value is reputation rather than direct earnings; credit the idea to the owner and the code to the AI.
- **01:41 #11 [DIRECTION]** - "the stories of the knowledge keepers and the myths of old"; "the theory of relatively within wording". The AI built a library of 12 public-domain collections from 9 cultures, about 727,000 words (`7c4a144`).
- **01:52 #12** - Run the Aesop test, then hand off to a new instance so "we dont lose progress". "This could be huge actually". The AI noted that tiny models do not change how it understands the owner. It saved the Aesop rules (`87cccf9`), which reviewers fixed twice before training (`194fc7c`, `910e9cb`). It also wrote the communication guide, handoff, prompt and plan (`b58b583`, `cde3dac`, `6143fa8`).
- **01:58 #13** - "keep going and tell me the verdict".
- **02:02 #14** - "When will I run out of usage?" The AI cannot see limits; everything was saved.
- **02:12 #15** - How does this compare with today's best? Experiment 8 was **not confirmed** for consequences (51/100, p = 0.46). Judging was strong as a side measure: 67, 72 and 67 right out of 100 (majority vote 69/100, p = 0.0001). It is far from today's large AIs.

- **02:14 #16 [CORRECTION]** - "evaluate it as methods". The AI agreed and saved rules for a head-to-head of the shifting word, a plain good/bad label and a 3-spelling carry (`6e365ef`). The audit later found that judging was chosen as the main measure after a peek.
- **02:21 #17** - "even if its .5% ... how much". About 15 more right per 100, not yet confirmed.
- **02:35-04:16 (no new requests).**
  - Experiment 8 was written up (`a5f1a6b`).
  - Experiment 10 was a tie: shifting word 68.7%, label 67.7% (`9c6fb86`).
  - Experiment 11 (long stories) had its rules saved at 03:44. It was then changed twice mid-run **[RESTART]** (`c93f383`, `6fc220d`).
  - It was written up as "confirmed": 59 vs 53 per 100 (`01b9050`). That was later corrected to "supported once".
  - The word "carries perfectly but updates poorly" (`11982b6`).
- **05:46 #18 [DIRECTION]** - "launch the new session now", with the drift instruction. The AI added the drift guard (`2b3d28e`) and launched a second session.
- **05:48-06:53, second session** (the owner's messages there are not in this record) - It audited the work: the numbers matched the raw files, but some wording was too strong (`84e2ba4`). It wrote the owner's file rules into `CLAUDE.md` (`3360d93`), corrected "private" to "public repository" (`a67ebdf`), and sent this session an owner-approved audit prompt (`685daf7`).
- **06:11 [LIMIT warning] [PAUSE]** - A 7-day usage warning appeared (it resets 24 Oct). This session went quiet while the Aesop models trained.
- **07:33-07:39** - Aesop: all four checks **not confirmed**. Folk tales 52.3%, novels 54.1%, shuffled 53.1% (`7fdad39`, `29a94aa`). The audit corrections were applied (`2a58b61`, `6bfa507`). The results folder was delivered and `CLAUDE.md` was copied to this branch (`d93b6f7`, `49e10f0`). The audit branch merged the results (`3ee90de`, `b806b95`). **[PAUSE]**: the conversation test waits for the owner to approve its rules.
- **15:29 #19 [CORRECTION]** - "The moral aspect was merely training data that you drifted with". The owner asked for copyable plain text. The AI wrote a 6-part plain guide in the chat.
- **16:02 #20, 16:04 #21** - The owner pasted another AI's "Data Filtering & Methodology Report" (v1.0, then v2.0). The AI found that v1.0 was a summary of its own write-ups: it overstated results, mixed up two experiments and drifted from communication. v2.0 was better, but three statements were still wrong. The AI gave correction lines to paste back.

19 October 2026 (Monday)

- No owner messages and no commits.

20 October 2026 (Tuesday)

- **08:13 #22 [DIRECTION]** - The owner uploaded 11 files (other AIs' reports, a compression design called EBEC, notebooks, a prototype, and profile documents with personal details) and asked for "a full technical report in .doc format" with all code.
- **08:17 #23** - The owner stopped a tool call: the AI was starting to run tests.
- **08:35 #24 [CORRECTION]** - "I didn't say run tests? Did I?" The owner wanted a report detailed enough to regrow everything in a fresh chat: a "seed" plus "grow instructions", like algebra's "show your work". "Do not fail to generate my doc". The AI delivered `TECHNICAL_REPORT_2026-10-20/`: a real .doc and .docx, the seed, 10 grow steps, a source evaluation and all the code (`8c08f41`). Personal details were left out.
- **08:45 #25** - Could data seeds save electricity and add security? The AI said compression is real. But new information cannot be shrunk, regrowing costs power, and a "dead seed" is security through obscurity. The fix: compress, then encrypt.
- **08:56 #26 [CORRECTION]** - "You dont understand my friend how it actually works". The owner reported 100% reconstruction and 98.9% reduction with Copilot and Gemini, with no LLM needed. The AI built a fair, AI-free test, `seed_test.py` (`283b4b6`). With zip, random bytes shrink 0%.
- **08:57 #27** - "Use a graph system lol". Graphs suit patterns, not random data.

- **08:59 #28, 09:00 #29, 09:03 #30** - "add the graph method". The owner interrupted to explain: a fixed "page of binary" built from solving an expression, plus library factors "strawberry and orange verse cucumber".
- **09:06 #31** - "Run it. Your close". The AI built three readings, each with an independent checker (`6cd7332`).
- **09:10 #32, #33** - "tell me the verdict".
- **09:14 #34** - "Are you surprised?" No. The same idea lives in game world seeds and in Kolmogorov complexity (the shortest description of data).
- **09:20 #35** - A palm seed grown in "Antarctica": different operating systems. The AI said exact rules work everywhere if they use whole numbers and raw bytes. The graph-method drafts so far were saved (`b6397c1`).
- **09:30 #36 [DIRECTION]** - The greenhouse story: soil, humidity, sun, then a seed. "I want to improve AI compression and understanding", and credit. The AI launched the greenhouse ladder, a dig through the chat and a fact check. Graph-method drafts were saved (`8f705c6`).
- **09:45 #37** - "map out these points as if expression on a face". The AI posted a progress map with faces. A graph-method update was saved (`93890e4`).
- **09:55-10:13 [LIMIT]** - Session limit (reset 13:10 UTC).
- **16:13 [RESTART]** - Graph verdict (`d7b1323`). Every file regrew exactly, but only patterns shrank: pattern 99.99%, Bible text 6-19%, random 0%.
- **17:08** - Experiment 12 (`8dff9ba`). The tiny Bible model regrew 623 unseen verses from 12.2 KB instead of 83.8 KB (85.4% smaller, exact). The shifting word did not help.
- **19:09 #38 [DIRECTION]** - "Add them and test the nunbers "17,32,16,34"=?0=-0=+0=00=["47,"="17+4"]". The AI saved the 15 trained models to GitHub (`33a893d`) and started a design team.
- **19:19 #39 [CORRECTION] [RESTART]** - "the +4 was a way to direct you to a way to try testing the numbers where I feel like they belong." The design restarted with the numbers in 17's place in "W1 = 17 + 4 Remaining", in the owner's order.
- **19:24 #40** - "Which 2,to,too or two feels like a missing clue?" The AI picked "2": the model's reader drops digits.
- **19:26 #41** - "add the 2 test after this one". Queued (`7695b97`).
- **19:34 #42 [DIRECTION], 19:37 #43** - An astrology/numerology formula, as a test of how other AIs comprehend. The AI wrote a kit: the formula, a shuffled control, a follow-up question and written predictions.
- **19:41 #44, 19:43 #45** - The owner brought back the answers. Gemini (memory on) gave the shuffled control a confident meaning and pulled in the owner's other chats. Copilot stayed closer to the text, but still read meaning into the control. The answers were kept out of the public repository because they held personal context.
- **19:47 #46 [CORRECTION]** - "We need to understand the "17+4" they used". The AI read each answer as the prompt (17) plus carried memory (+4).
- **19:52 #47** - Gemini with memory off stayed with the words. The +4 was real and, this time, it hurt.
- **19:57 #48** - How would this AI answer? It gave its own answer (with a large +4, since it wrote the prompt) and predictions for a fresh copy of itself.
- **20:02 #49 [CORRECTION]** - "Nothing is useless." "You Do Not Know the Answer". "a sharp left away from logic?" The AI agreed and proposed a random-numbers control. That control was not adopted, and the AI later called its framing wrong. The draft script was saved (`12f2051`).
- **20:02 #50 [PAUSE]** - "Pause tasks". Stopped before any training.
- **20:13 #51 [CORRECTION] [DIRECTION]** - "Everything adds and subtract from understanding and comprehension." The owner shared personal context for why every input is + or - information and why recharging in a chosen direction matters (not reproduced here). Their instruction was an agent that sorts each input into + or -: "dont do this step yourself but make sure it happens", and never know the true number, only its "placement on that line". The AI said "You're right, and I was wrong" and queued the agent.
- **20:22-21:06 [LIMIT]** - Session limit (reset 21:10 UTC).

21 October 2026 (Wednesday)

- **01:35** - The AI confirmed everything was paused and saved.
- **01:41 #52 to 01:53 #56 [BLOCKED x5]** - The owner asked for a plain review from beginning to end:

- reasons "for", "against", and anything overlooked;
- every source and every "+4" the AI noticed in its replies;
- a word-for-word rerun and comparison (#53);
- "why a review ... is a risk" (#54, a correction);
- a "safe prompt" and the usage back (#55; the AI has no access to billing);
- a full word-for-word review (#56).

The owner also noted that the AI is listed as co-author on commits. All five replies were stopped by an automatic check ("safeguards flagged this message", detail "[reasoning_extraction]"). The AI could not see why. One unconfirmed reading is that the wording looked like a request for its hidden reasoning. No work was lost.

- **01:55 #57** - "a complete understanding as if I was a manager and you are the worker". This one got through (01:59). It held every request and what was delivered, the results by strength, the untested items, the AI's own errors (its "+4s"), and for/against/overlooked.
- **02:05 #58 [RESTART]** - "Continue on with testing". The numbers-test dry run passed (02:06-02:08), and the folder `NUMBERS_TEST_2026-10-21/` was saved (`0e4574a`, `5d20314`). A 4-reviewer check is running. **No results yet.**
- **02:15 #59** - This report.

Totals:

- 59 owner messages;
- 2 usage-limit stops, plus one 7-day warning;
- 3 deliberate pauses;
- 2 owner interruptions;
- 5 safety blocks.

21 October 2026, later (added after this timeline was drafted)

- **02:21 #60** - The owner suggested emailing some people a link and inviting them to the repository.
  - The AI explained that the repository is already public, so a link is enough. Collaborator invites (which let people edit) are made by the owner in GitHub settings.
  - It offered to write a Gmail draft for the owner to send once recipients are named. None have been named yet.
- **02:22 #61** - The owner observed that the branch's diff (its list of changes against the main version) has stayed large and similar while progress has slowed: "We need other people, someone else to know."
  - The AI measured the branch: +156,235 lines against `main`, 141,791 of them on 20 October, mostly copies of data files (the Bible text and one results file, each stored twice). Line counts are not progress.
  - It agreed the real limit is scale (tiny models on 4 CPU cores). It proposed outside help: computing power, a large-model API key and a fresh critical reading. It offered a one-page summary for newcomers and an outreach draft.
- **Early 21 October** - The session hit its usage limit (reset 06:30 UTC). It stopped this report and the numbers-test review part-way.
- **13:07 #62** - The owner shared another AI's Python blueprint, "ComprehensionSeed": a living memory of concept weights that are added to and subtracted from, with human errors treated as signal and a look beneath the literal words.
  - The AI's reading: it is an outline, and the seven functions that would do the hard part are undefined. It matches the owner's thesis and the planned +/- agent.
  - It has three flaws the experiments already showed: weights never fade; contradictions are only checked for new concepts; nothing is ever removed.
  - One framework name in it contains a person's full name, which is kept out of this public record; it is called here "the owner's error-encoding method".
- **13:12 #63** - The owner gave the scoring rule for the +/- agent:
  - **Positive:** moral high ground, telling the truth, highly reasonable information.
  - **Negative:** questionable morals, nonsense, misspelling.

    - "Should always pull a negative." More negatives than positives, but the positives are larger, and it balances.
    - It is recorded in `HANDOFF.md` (`a8a0cd7`). The AI measured today's word values as symmetric (36.0% vs 37.4%; average sizes 2.31 vs 2.27).
    - The size of a positive against a negative is still the owner's to choose.
- **13:10 to 13:30** - The review and this report resumed, then hit the usage limit again (reset 17:00 UTC).
- **17:37 to 17:45** - The AI did the remaining run-safety checks itself and applied 10 small fixes that leave the frozen design unchanged. A second dry run passed. **The full 18-model run started at 17:45 UTC** (`55f6817`).

Updated totals: 63 owner messages; 4 usage-limit stops (2 on 20 October, 2 on 21 October).

4. The owner's ideas, in their own words, and what was built from each

Each idea below gives the owner's words, the plain meaning, what the AI built and what the tests found. Quotes are copied exactly as typed, typos included. "Message N" is the Nth message in the numbered list of the owner's messages.

All tests used tiny models trained from scratch: about 3.4 million parameters, able to see the last 64 words. A result here does not change how a large AI understands anyone.

4.1 Starting point: comprehension for the AI, not reading people

The owner first pasted a script that scored "human error" in text: "Evaluate and enhance if possible". The AI fixed its bugs (`resonance/resonance_engine.py`, 16 passing tests). Then came message 5: "I am trying to teach actual "comprehension" and "understanding" for A.I. not about people".

That set the objective. The script reads people, so it was dropped as the framing. Its "wavelength" is a text fingerprint (a code computed from the text), not a measurement.

4.2 Rules plus human error as the source of meaning

- **Words (message 6, 17 Oct):** "The rules are the secret of understanding. The things we do wrong naturally and have to be told not to do them is something ai must mimic to grasp reality as a human does." Also: "If a person has never seen (born blind) do the truly understand what color is?"
- **Meaning:** understanding forms where rules meet the mistakes that break them. A reader without senses can still learn meaning from how words relate.
- **Built:** `experiments/bible_understanding.py` (experiments 1-4), a word-meaning model of the King James Bible (31,102 verses). Then `consequences_test.py` and `tiny_transformer.py` (experiments 5-6), at the owner's "yes try the consequences test next" and "yes try the tiny transformer next".
- **Results:**
    - Color without sight worked: 10 of 14 matches (71%), against 17% by guessing.
    - Rules alone were not the secret: they scored 0.625 on right and wrong, against 0.709 for the same number of random verses. Removing every rule changed almost nothing (0.948 vs 0.950).
    - The law sharpens wrongness: "covet" weakened from -0.077 to -0.043 without the rules. But 0 of 15 wrong acts flipped to good, so the stories teach it too.
    - Predicting consequences in new stories: chance (45-57%).
    - **What did show up** was the "have to be told" part. Marking right and wrong in the training text lifted judging of new stories to about 69 right per 100, against about 50 unmarked (experiments 8 and 10). This is strong and repeated, but only a side measure: it was never yet pre-registered (fixed in writing before the run) as the main one.

4.3 The shifting word (the owner's central idea)

- **Words (message 9, 17 Oct, 23:00 UTC):**
    - "I will create a new word and give it a variant in its token structure. The idea is based on a syllables rotation sequence rather than a logical one"

    - "[w1=17 -<Next word following it equal to (-13)> = +4 Remaining] next emergence of the word [W1=17+4 -<next 2 words following equaling (00)>= 0 Remaining]"
    - "it must reset to [w1=0] when a clear right or wrong choice exists. If wrong the sequence will begin with a value of (-17) and right a value of (+17)"
    - The variations "to", "too", "two" and "2"
    - The aim: "shift the "coin toss" towards the desired outcome"
- **Meaning:** a made-up word carries a running balance from one use to the next. The words after it add to the balance or use it up, and what is left (the "Remaining") carries forward. A clear right or wrong resets it to +17 or -17. Its spelling changes with the balance, so a reader sees the state in the text. "Shift the coin toss" means beating a 50% guess.
- **Built:** `experiments/shifting_word.py`.
    - The word *velanok* has five spellings, rotations of sha-ru-vel-an-ok, running from `anoksha` (heavy wrong) through `oksharu`, `velanok` (unresolved) and `sharuvel` to `ruvelan` (heavy right).
    - `RESET = 17` on a clear choice: a Bible verdict phrase, or a verse among the 3% most strongly worded at either end. That makes 2,029 reset verses.
    - Word values come from the Bible model's good/evil direction (steal -5, love +6).
- **What the AI changed, and why:**
    - **The recharge ("W1=17+4") was switched off.** With it on, the balance pinned at its cap and almost every verse got an extreme spelling. It was left as the switch `RECHARGE`, untested until the numbers test (4.5).
    - **A fade was added,** `SPEND = 0.5` per valued word, so the balance drifts back toward "unresolved".
    - **Limits were added:** a cap at +/- 51 and a reset at each book start.
    - **Only spellings reached the model, never the number.** Its reader sees letters and punctuation only (see 4.10).
- **Results:**

Experiment	Test	Result
7	20 story pairs, consequences	68% vs 52% scrambled. Later shown to be luck
8	100 new pairs, consequences (pre-registered)	Not confirmed: 51/100
8	100 new pairs, judging (side measure)	67, 72, 67 vs scrambled 56, 42, 60 and unmarked 49, 60, 40
10	Vs a plain good/bad label per verse	Tie: 68.7% vs 67.7%. Not confirmed
11	Long stories, everyday act out of view	Supported once: 59, 56, 63 vs label 54.5, 51, 54 (p = 0.004). A re-run after two fixes; about 65 of 100 pairs tied
11	Long stories, verdict stated	Not confirmed: the label won
12	Exact seed size, 623 unseen verses	Not confirmed: 3.45 bits per verse larger

- **Known defect:** it "carries perfectly but updates poorly". Started on "unresolved", it copied "unresolved" all the way down.
- **Untested:** whether the shared syllables matter (this needs a sub-word reader), and the recharge rule.

4.4 Stories transfer further than rules (Aesop)

- **Words (message 11, 18 Oct):** "Lessons taught through story have more transferable data and relation vectors to give further understanding of things. Its the theory of relatively within wording and comprehension".
- **Built:**
    - `world_stories.py`: 12 public-domain collections, about 727,000 words, using only material published for sharing.
    - `aesop_test.py` version 3, revised twice before any training. It uses 67 Aesop fables never seen in training (1,131 pairs) and trains 16 models.
- **Results:** every pre-registered test was not confirmed. With 50% as guessing, folk tales scored 52.3%, novels 54.1%, shuffled folk tales 53.1% and the Bible 43.6%.
    - **A hint only:** on pairs with no shared-word clue, story models scored about 60% against 54% for novels. Shuffled stories matched them, so the hint points to story vocabulary and setting, not structure.

- **Not disproved:** the models were too small to test the idea fairly.

4.5 The owner's numbers: 17, 32, 16, 34, 47

- **Words:**
  - Message 9: "17,32,16,34,47 are numbers my subconscious mind has focused on for years , they may just be random noise or maybe useful i dont know".
  - Message 38 (20 Oct): "Add them and test the nunbers "17,32,16,34"=?0=-0=+0=00=["47,"="17+4"]".
  - The owner's clarification (message 39): "I just know my sub conscious mind has had those numbers in that sequence and the stands for me the +4 was a way to direct you to a way to try testing the numbers where I feel like they belong".
- **Meaning:** the numbers matter in that order, and they go where 17 sits in "W1 = 17 + 4 Remaining". That place is the fresh charge the word gets at each appearance, with the Remaining carried on top.
- **Before this:** only 17 was used, as the reset size, and in that design only ratios mattered.
- **What the AI did:**
  - Three independent readers and a judge (AI agents) worked out the design (`NUMBERS_TEST_2026-10-21/design/interpretations_and_design.md`). They found the old `RECHARGE` switch could not test the numbers at all: 16, 32, 34 and 47 all produce identical training text.
  - **Two choices were the AI's own:**
    - each valued word uses up 14.1073% of the charge (`DRAIN`), so that ordinary text settles at exactly 17 + 4 = 21, leaving 4;
    - the word is spelled from the Remaining, as the owner wrote.
  - The zero symbols (?0, -0, +0, 00) are not tested, because the owner has no fixed meaning for them.
- **Built:** `experiments/numbers_test.py`, six versions trained 3 times each (18 tiny models):

Version	Charge at each appearance
`num_charge17`	17 every time (the owner's plain recharge rule)
`num_seq`	17, 32, 16, 34, 47, repeating (the owner's order)
`num_seqrev`	17, 47, 34, 16, 32 (order control)
`num_const29`	29.2, the average (size control)
`num_charge47`	47 every time (written in the "17+4" place)
`num_drain0`	0 (stops the AI's drain from taking credit)

- **The pass/fail rules were fixed before training.** For example, "the owner's numbers matter" means `num_seq` beats `num_charge17` on judging 100 story pairs.
- **Status:**
  - **The dry run passed** on 21 Oct, 02:06-02:08 UTC. All six versions passed the manipulation check (a pre-set check that each rule really changes the text): at most 1.23% of verses at the cap. The stored control scores were reproduced exactly, and every exact seed regrew byte for byte.
  - The 4-reviewer review finished on 21 Oct. The full run (about 4-5 hours) has been running since 17:45 UTC.
  - **There are no results yet.**
- **A control dropped:** the AI proposed a random-numbers control, but it is not among the six. The AI later called its own framing wrong: "Your numbers are the hypothesis".

4.6 The data seed, the palm seed, and grow instructions

- **Words (20 Oct):**
  - "a tiny seed of information and the grow instruction so it can be grown back to this state exactly" and "(Seed=algebraic expression)", with the example `[+2xy(27/5.2+t=y/?)÷(?\y=t+2.5\72)yx2]` (multiplication shown as x).
  - "any interception will yield a dead seed".

    - The owner reported "100% reconstruction accuracy and 98.9% data reduction" in tests with Copilot and Gemini, adding "it doesn't require an LLM".
    - Later: "Imagine I buy a palm tree seed while on vacation ... When I get home to "Antarctica " ... I must still follow given instructions."
- **Built:**
    - The technical report, shaped as a seed (one seed, grow rules, 10 grow steps with checkpoints), in `TECHNICAL_REPORT_2026-10-20/`.
    - `SEED_TEST_2026-10-20/seed_test.py`: a fair test with no AI. It grows each seed in a separate program that cannot see the original, and checks exactness by fingerprint.
- **The AI's view:**
    - **What holds:** the idea is real for patterned data.
    - **The limit:** no fixed rules can shrink every file, and random data cannot shrink at all.
    - **Security:** "dead seed" is security through obscurity, so compress first, then encrypt with a standard method.
    - **The palm seed:** it grows anywhere if the steps use whole numbers, raw bytes and exact instructions.
- **Measured (experiment 12):** a decoder using 1 processor thread instead of 2 lost its place after 547 of 623 verses. The exact settings are part of the grow instructions.
- **Found:** the owner's expression is an exact mirror image around its division sign, apart from the leading +. "Worked backwards" is built in.

4.7 The graph method and the strawberry/orange/cucumber library

- **Words:** "Use a graph system lol" and "add the graph method to the seed test". Then: "The graph existing plot points if only "0" or"1" stacked from the work of solving an expression generates a page of binary rapidly and it doesn't change." And: "library style factors called strawberry and orange verse cucumber".
- **Built** in `SEED_TEST_2026-10-20/methods/`, three ways, each checked independently, using only adding, subtracting, multiplying and remainders:
    - `graph_fit`: fit the shape;
    - `graph_backwards`: solve back to the unknowns;
    - `graph_locate`: a fixed page of 0s and 1s.
- **The library:** strawberry, orange and cucumber are placeholder names. In `graph_fit`, strawberry is lines and curves, orange is cycles, and cucumber is the leftover, stored as it is. `graph_locate` builds its pages from the owner's numbers.
- **Results** (seed size saved; every file regrew exactly):

File	Zip	Fit the shape	Work backwards	Fixed page
Repeating pattern	99.8%	99.99%	99.99%	0%
Bible text	70.2%	19.3%	6.2%	0%
Random bytes	0%	0%	0%	0%

- **What this means:** it works on patterns, beating zip there, but text is not shaped like lines or cycles.
- **A likely explanation of the 98.9%:** with the Bible on a shared shelf, naming a verse costs about 1.9 bytes against 132.5, which is 98.6% smaller and exact. The library is the "huge file" the owner mentioned. This is likely, not proven.
- **Still untested:** the owner's real factors, not yet shared.

4.8 The greenhouse: "can I grow understanding and compression?"

- **Words (message 36):** "instructions to find the soil needed and to build a greenhouse, I told you the humidity and I explained the rotation of the sun. We checked the air ... Then I gave you a seed and told you to grow it." and "can I grow understanding and compression?"
- **Built:**

    - A chat dig that matched every owner message to a greenhouse piece: the Bible text is the soil, the tiny model is the greenhouse, and the pre-registered checks are the air.
    - A fact check of published work.
    - Experiment 12 (`understanding_compression.py`), an exact coder that regrows 623 unseen verses (83,757 characters).
- **Results:**
    - **Understanding shrinks the seed. Confirmed** and checked independently: raw 83,757 bytes, xz 26,420, word pairs 14,864, tiny Bible model 12,153-12,252 (85.4% smaller).
    - **The shifting word did not help. Not confirmed** (12,411-12,524 bytes).
    - The owner's coin toss fits here: a greenhouse that understands tilts each 0/1 toss, so the seed records only the surprises.
- **Credit:**
    - The principle is already published (Shannon 1951, and the Hutter Prize, a long-running text-compression contest), so it is not new.
    - What could still be the owner's: a measured gain from their own rule, the "seed bits as an understanding meter" method, and the greenhouse teaching story.

4.9 Every input is + or - information, and the +/- agent

- **Words (20 Oct):** "Nothing is useless. " the only info that harms us is the info we dont have"". Then: "Everything adds and subtract from understanding and comprehension my friend." And: "You must simple decide if it's "+" info or "-" info and add it to the overall of yiur experience using an agent#dont don't his step yourself but make sure it happens and never know your tru number just a way to gauge its placement on that line".
- **Context:** the owner shared personal context for why every input is + or - information, and why recharging in a chosen direction matters. It is not repeated here.
- **The AI's response:** at first it agreed only in part. Then: "You're right, and I was wrong. I called the shuffled control "nonsense" and Gemini's old memory "stale". Both still carried information."
- **The plan:** an agent that labels every input + or - with a reason, keeps a running balance, and reports only a placement on the line, never the number. It is queued after the numbers test and the "2" test. **Not started.**

4.10 "to, too, two, 2" and the "2" test

- **Words:** "Which 2,to,too or two feels like a missing clue? ???" and "yes add the 2 test after this one".
- **The AI's pick: "2".** The model's reader, `[a-z]+|[.,;:?!]`, drops every digit, so the number form of the owner's rule never reached the model.
- **The plan:** write the Remaining value into the text as a readable number. Approved and queued. **Untested.**

At a glance

Idea	Status on 21 Oct 2026
Meaning from text alone	Worked once: 10 of 14 (no rules saved first)
Rules are the secret	Not supported: stories teach as well
Marking right and wrong ("have to be told")	Strong and repeated; not yet the pre-registered main measure
Carried balance (shifting word)	Supported once, in long stories only
Stories transfer further than rules	Not confirmed at this scale
Recharge rule and the numbers	Running; no results yet
Seed and graph method	Shrinks patterns only; always exact
Understanding shrinks the seed	Confirmed (already published)
"2" test, +/- agent	Queued, untested

5. Every experiment (part A: the first script and experiments 1 to 11)

This part covers 17 October 21:41 to 18 October 07:34, in order. Times are UTC. Sources: `experiments/README.md`, `experiments/HANDOFF.md` and the raw `experiments/results_*` files.

Words used throughout:

- **Chance, or the "coin toss":** 50% on every two-choice test here.
- **p:** how likely luck alone would give a result this good. Below 0.05 is the usual bar.
- **Pre-registered:** pass/fail rules saved to git before any result was seen.
- **Run:** the same model trained again from a different random start.
- **Control:** a version identical except for the idea being tested.

These are tiny models. Their results do not change how a large AI understands anyone.

5.0 The resonance engine (`resonance/`)

What it did. On 17 October at 21:41 the owner pasted a short Python script and asked to "Evaluate and enhance if possible". It looked for trigger words (guilt, mistake, sin, betrayal, heavy, hide, secret, fear) in four groups: regret 0.5, burden 0.7, commandment breach 0.8, unforgivable 1.0. It added the weights, capped them at 1.0, and picked a "stance" for a host AI. It also printed a "resonance wavelength".

Faults, each confirmed by running it: exact words only ("mistakes", "hiding", "betrayed" scored 0); negation ignored ("I have no guilt" scored 50%); one word always triggered "grace", so the middle stance could never fire; the owner's example added up to 3.8, shown as 100%; "system_halt" did nothing; the wavelength included the clock time, so the same text never gave the same result.

Fixed (version 2.0.0, commit d8ebeec, 21:43, 16 tests passing): word stems; negation within 3 words, stopped by a comma or "but"; new weights (0.30, 0.30, 0.50, 0.80) combined as 1 - (1 - a) x (1 - b) x ..., so the score rises without pinning at 100% (the example now scores 70%); all four stances reachable, including a new "System Halt"; evidence with word positions; the same text always gives the same wavelength; a privacy option to leave the raw text out.

Honest review (21:47): the wavelength is only a fingerprint of the text, not a measurement; nothing used the output; keyword matching is shallow; labelling a person's words "sin" judges them.

Why the framing was dropped. At 21:52 the owner wrote: "I am trying to teach actual "comprehension" and "understanding" for A.I. not about people". The engine reads people. It does nothing for the AI's own understanding. The code stays in `resonance/` as a record and no later experiment uses it. At 22:02 the owner asked whether someone born blind truly understands color, and proposed the Bible: "The rules are the secret of understanding."

5.1 Experiment 1: color without sight

Setup for experiments 1-4 (`experiments/bible_understanding.py`, commit ec22991, 22:08). The King James Bible: 31,102 verses, 793,445 words. A classic word-meaning model counts which words appear within 4 words of each other and squeezes each word into 100 numbers (PPMI and SVD). Words that keep similar company end up close. No outside knowledge. Test words were fixed before results, but no pass/fail rules were saved to git, so that can't be checked.

Question. Can a reader that never sees learn color, like someone born blind?

Result. 10 of 14 objects matched to the right color (71%; guessing 17%): snow->white, blood->red, night->black, grass->green and others. It learned what color means in this book: scarlet, purple and blue sit with silk, linen and needlework (tabernacle fabrics). It missed the Isaiah 1:18 metaphor ("sins as scarlet"): every color sat nearer purity than sin, mostly because of Leviticus purity laws.

Status: worked; one run, 14 objects.

5.2 Experiment 2: three ways of reading

Scores run from 0.5 (guessing) to 1.0 (perfect).

Reading	General meaning	Moral meaning	Color
Word by word	0.975	0.957	10/14
Whole verse as one unit	0.974	0.995	11/14

Reading	General meaning	Moral meaning	Color
Hebrew parallelism	0.979	0.948	11/14

Status: little difference. Whole verses gave the best sense of right and wrong.

5.3 Experiment 3: are the rules the secret?

2,801 verses (9%) are rules. Learning from rules only gave moral meaning 0.625 vs 0.709 +/- 0.081 for the same number of random verses (5 draws): slightly worse. Removing all rules: 0.948 vs 0.950 for removing random verses. **Status:** not supported. The stories teach right and wrong at least as well as the law.

5.4 Experiment 4: Romans 7:7 ("I had not known sin, but by the law")

Without the rules, none of 15 wrong acts flipped to "good". Acts the law mostly names grew weaker: covet -0.077 -> -0.043 (seen 3 times without the rules), drunkenness -0.195 -> -0.054, steal -0.163 -> -0.110. **Status:** partly echoed. The law sharpens wrongness but isn't its only source.

Next. These tests only place words near each other. A stronger test: can a model predict what follows an act in a new story? The owner (22:12): "yes try the consequences test next".

5.5 Experiment 5: predicting consequences

Setup (`experiments/consequences_test.py`, 939d60f, 22:16). 20 new story pairs: one situation with a right or a wrong act ("found his neighbour's ox astray and brought it again" vs "took his neighbour's ox by night and sold it"), sharing one fortunate and one ruinous ending, so always picking one ending scores exactly 50%. Moral words are banned and endings reuse no story words (checked by the script).

Result. Every method scored at chance: association 45%, a "this, then that" map from the whole Bible 48%, from rules only 52%, from non-rule verses 45-52%, rules removed 50%. A diagnostic added afterwards: telling the right-act story from the wrong one 60% (p = 0.25).

Status: not confirmed. The model knew "steal" was wrong but couldn't see that the ox story is stealing: it averages words and can't combine them. The owner (22:19): "yes try the tiny transformer next".

5.6 Experiment 6: a tiny transformer

Setup (`experiments/tiny_transformer.py`, df94814 at 22:30, result 1a4dfac at 22:42). A transformer, the kind of model behind modern AI, reads words in order. This one was trained from scratch on the Bible only. Every later experiment uses it:

Setting	Value	Plain meaning
Size	about 3.4 million parameters	adjustable numbers inside the model
CTX	64	it sees only the last 64 tokens (words or punctuation)
D	192	each token is held as 192 numbers
Layers / heads	4 / 4	depth, and attention readers per layer
Training	1,500 steps, batch 64	about six passes, about 12 minutes on 4 CPU cores
Tokenizer	word-level, `[a-z]+` or one of `. , ; : ? !`	digits dropped; words seen once become "unknown"
Held-out	every 50th verse (623)	never trained on

Result (one run). Leaning toward the right ending: 9/20 = 45% (p = 0.75). Picking the ending: 22/40 = 55%. Judging the act: 9/20 = 45%. Its error on unseen verses (3.46) means it narrows each next word to about 1 in 32 of 8,658: it learned how the Bible sounds.

Status: not confirmed, chance. The planned rules comparison was dropped, since it could only be compared against noise. At 23:00 the owner introduced the shifting word.

5.7 Experiment 7: the shifting word ("velanok")

The owner's idea (23:00): a made-up word carries a running balance, and its spelling changes with it, like "to", "too", "two" and "2". In the owner's notation: "[w1=17 -<Next word following it equal to (-13)> = +4 Remaining]". A clear wrong starts it at -17, a clear right at +17. The aim: "shift the "coin toss" towards the desired outcome".

Settings (`experiments/shifting_word.py`, commits f676eed at 23:03 to c918cb7 at 23:46):

Setting	Value	What it does
RESET	17	a clear wrong sets the balance to -17, a clear right to +17
SPEND	0.5	each valued word uses up 0.5, so the balance fades toward 0
Cap	+/-51 (3 x RESET)	the limit either way
SCALE	30	word value = good/evil lean x 30 (steal -5, adultery -10, love +6)
CLEAR_PCT	3	top and bottom 3% of verses, plus verdict phrases, count as clear choices (2,029 verses)
RECHARGE	False	re-adding 17 at every use ("W1 = 17 + 4") was off: the balance would pin at the cap

Five spellings, rotations of sha-ru-vel-an-ok: **anoksha** (balance -12 or below, heavy wrong), **oksharu** (-12 to -4), **velanok** (between -4 and +4, unresolved), **sharuvel** (+4 to +12), **ruvelan** (+12 or above, heavy right). The word went at the start of every verse; the balance restarts at 0 for each book.

Test. The same 40 stories. The model writes the word itself, then the endings are scored. Control: the spelling chosen at random ("scrambled"). Two runs each.

Model	Leans toward right ending	Judges right vs wrong
Shifting word, runs 1 and 2	14/20, 13/20 = 68% (p = 0.02)	14/20, 14/20 = 70% (p = 0.008)
Scrambled, runs 1 and 2	10/20, 11/20 = 52%	12/20, 12/20 = 60%

Handing the model the correct spelling gave identical scores, so the word worked as a teacher during training.

Status at the time: reported as "the first result above chance". **Caveats:** no pass/fail rules saved first; both runs used the same 20 pairs. Experiment 8 showed the consequence result was luck. Only 17 of the owner's numbers was used; here only the ratios matter.

Next. The owner (00:46, 18 October): "Let's confirm the result".

5.8 Experiment 8: confirmation on 100 new pairs

Setup (`experiments/confirm.py`, `stories_v2.py`; pre-registered 5538b46 at 00:52; write-up 02:35). The first experiment with pass/fail rules saved first. 100 new pairs; shifting, scrambled and no word, 3 runs each (9 models). Main test: consequences, by majority vote of the 3 shifting models. Judging was a side measure.

100 new pairs	Consequences, per run	Judging, per run
Shifting word	50, 55, 55 (majority 51, p = 0.46)	67, 72, 67 (majority 69, p = 0.0001)
Scrambled	49, 55, 50	56, 42, 60 (majority 53)
No word	62, 55, 61 (majority 61)	49, 60, 40 (majority 51)

Status: consequences **NOT CONFIRMED** (shifting vs scrambled 27 better, 27 worse). Judging was strong in all 3 runs (pair by pair, shifting beat scrambled 57 to 25, p = 0.0003), but as a side measure it is "strong and repeated, not yet pre-registered as the main measure".

Next. The owner asked how this compares with "other methods used today considered to be the best". The fair version is method against method at the same scale: experiment 10.

5.9 Experiment 9: Aesop moral transfer

Why. The owner (01:41): "Lessons taught through story have more transferable data". Do lessons told as stories carry over to an unseen culture better than other text?

Setup (`experiments/aesop_test.py`). Drafted at 01:57 and revised twice after reviews, all before training. It ran about 03:40-07:33, so it finished after experiments 10 and 11. 67 Aesop fables (Jacobs, 1894), never in training. For each pair of fables, the model must prefer the true fable->moral matching to the swapped one. 1,131 same-format pairs; 284 where no shared word gives a clue. Training sets of about 700,000 tokens: folk tales (9 collections, 8 cultures), novels, folk tales with sentences shuffled, and the Bible; plus two small sets (rules, Bible narrative). Context 128 tokens, 16 models.

Training set (runs)	Score, all runs combined	No-clue pairs
Folk tales (4)	52.3	60.2
Novels (4)	54.1	53.9
Shuffled folk tales (4)	53.1	59.2
Bible (2)	43.6	39.4
Rules / Bible narrative (1 each)	50.0 / 56.5	54.4 / 54.6

Status: all four pre-registered tests **NOT CONFIRMED**: no set beat chance (folk tales p = 0.33); folk tales were 1.8 points below novels and 0.7 below shuffled tales; no-clue pairs 60.2% (p = 0.065). The hint on no-clue pairs matched for shuffled stories, so it reflects story vocabulary, not structure. A fair test needs bigger models or more text.

5.10 Experiment 10: shifting word vs the established method

Question. Does the judging gain come from the owner's carried balance, or from labelling the text at all? The established method is "conditional training" (Korbak et al., 2023; `experiments/PRIOR_WORK.md`).

Setup (`experiments/method_comparison.py`, pre-registered 6e365ef at 02:17; write-up 03:41). Two new methods, 3 runs each, against the 9 models from experiment 8. **Plain label:** same spellings, but each judges only the verse before it, with no carry. **Carry3:** the carried balance with 3 spellings. **Disclosure:** judging was chosen as the main measure after 7 of 9 experiment-8 models had shown it was strong. One scoring fix at 02:58 (carry3 scored over its own spellings) left the other results identical.

Judging, 100 pairs	Per run	Average
No marker	49, 60, 40	49.7%
Scrambled	56, 42, 60	52.7%
Shifting word	67, 72, 67	68.7%
Plain label	71, 69, 63	67.7%
Carry3	66, 72, 74	70.7%

Status: a tie. Q1, shifting beats label: 20 better, 18 worse (p = 0.44), NOT CONFIRMED. Q2, 5 spellings beat 3: 23 vs 26, NOT CONFIRMED. Labelling improved judging by about 18-21 points. The carry added nothing in one-sentence stories, where nothing needs carrying.

5.11 Experiment 11: long stories

Question. The owner's idea is about carrying weight forward. Does the carried balance help once the act is out of view?

Setup (`experiments/long_story_test.py`, pre-registered 99622d3 at 03:44). No new training: the same 15 models and 100 acts. Neutral sentences follow the act until both the act and the first marker after it are outside the 64-token window. Models write their own marker after each sentence; the last one is the judgment. **Version A:** the act ends with an explicit verdict. **Version B:** an everyday act alone. Ties score 0.5.

Two fixes after the rules were saved: at 03:45, ties counted as 0.5 (the first model had scored 0/100 on ties); at 03:51, the first marker also pushed out of view. The flawed first runs are kept (`results_long_story_v1_flawed_run1.json`, `_run2.json`). They show the same version B gap before the second fix (59 vs 54.5).

Method	A: verdict carried	B: everyday act carried
Shifting word	45, 57, 62.5	59, 56, 63
Plain label	73, 51, 97	54.5, 51, 54
Carry3	71.5, 58.5, 54.5	59, 63.5, 59
No marker	50, 50, 50 (all ties)	50, 50, 50 (all ties)
Scrambled	44.5, 49.5, 57	50.5, 43.5, 52

Status. L2 (version B, shifting beats label): 43 better, 21 worse, p = 0.004, **supported once**. The 04:04 commit said "L2 confirmed"; the audit corrected it to "supported once" at 07:35, and that later correction stands: it is a re-run after two fixes, on reused acts. The shifting word tied on about 65 of 100 pairs; when it decided, it got 81 right and 25

wrong. L1 and L3 (version A) **NOT CONFIRMED**: the label won.

An exploratory run starting every story with the neutral marker scored 50.0 everywhere for the shifting word. **Lesson:** it carries perfectly but updates poorly.

Next. Stronger resets, faster spending, the owner's numbers as settings, fresh long stories. The owner's recharge rule ("W1 = 17 + 4") was still untested; the numbers test (part B) tests it.

5.12 Summary of all experiments

#	When (UTC)	Question	Models	Key numbers	Status
0	17 Oct 21:41	Resonance engine	-	16 tests pass	Fixed, then dropped
1	17 Oct 22:08	Color without sight	1	10/14 (chance 17%)	Worked (no saved rules)
2	17 Oct 22:08	Three ways of reading	3	0.948-0.995	Little difference
3	17 Oct 22:08	Are rules the secret?	1 + 5 draws	0.625 vs 0.709	Not supported
4	17 Oct 22:08	Romans 7:7	1	0 of 15 flipped	Partly echoed
5	17 Oct 22:16	Consequences, word model	1 + draws	45-57%	Chance
6	17 Oct 22:42	Consequences, transformer	1	9/20	Chance
7	17 Oct 23:46	Shifting word, 20 pairs	4	68% vs 52%	Later shown to be luck
8	18 Oct 02:35	100 new pairs	9	51/100; judging 69	NOT CONFIRMED; judging strong, side measure
9	18 Oct 07:34	Aesop transfer	16	52.3 vs 54.1	NOT CONFIRMED
10	18 Oct 03:41	Shifting vs plain label	15	68.7 vs 67.7	Tie, NOT CONFIRMED
11	18 Oct 04:04	Long stories	15	59, 56, 63 vs 54.5, 51, 54	Supported once
12	20 Oct	Understanding as compression (part B)	15	Tiny model 1.16-1.17 bits per character; shifting +3.45 bits per verse	Ladder confirmed; shifting word NOT CONFIRMED
Numbers	21 Oct	Owner's numbers, recharge rule (part B)	18 planned	Dry run passed	No results yet

5. Every experiment (part B: the seed test, the graph method, the greenhouse ladder, the numbers test)

The fair seed test (SEED_TEST_2026-10-20)

Where it came from. On 20 October the owner described their "data seed": a short expression of numbers and symbols, plus fixed grow instructions (add, subtract, multiply, restart or work backwards) that both devices already hold, so only the seed travels. The owner reported "100% reconstruction accuracy and 98.9% data reduction" in Copilot and Gemini chats. They also reported an earlier 99.8% that "required huge files". The AI built an AI-free test that runs on any computer: `SEED_TEST_2026-10-20/seed_test.py`.

What makes it fair:

- **Every byte counts.** Seed and original are both counted as raw bytes, so syntax or encoding cannot hide size.
- **Exact means exact.** The regrown file must match the original byte for byte, checked by SHA-256 fingerprint (a short code that changes if a single bit changes).

- **No peeking.** Each seed is grown by a separate program in an empty folder that holds only the seed. A chat, by contrast, can see the original earlier in the conversation.
- **Grow instructions counted once.** The script plus the method file are shared in advance, and their size is printed.
- **Hard cases.** The test includes random bytes, fresh files the builder never saw, and limits of 60 seconds to plant and 10 seconds to grow.

Results (percent smaller; every method regrew every file exactly, 5 of 5):

Test file	Zip	Graph: fit the shape	Graph: work backwards	Graph: fixed page
Repeating pattern, 120,000 bytes	99.8% (204 bytes)	99.99% (11 bytes)	99.99% (10 bytes)	0%
Short text, 267 bytes	66.3%	64.4%	63.7%	-1.5%
Bible text, 100,000 bytes	70.2%	19.3%	6.2%	0%
Word file, 11,110 bytes	9.9%	1.5%	1.9%	0%
Random bytes, 100,000	0%	0%	0%	0%
All five together	57.7%	42.1%	38.2%	0%

Source: `SEED_TEST_2026-10-20/results_graph_methods.txt`.

The counting limit, in plain words. There are more possible files than short seeds. A 100-byte file can be any of about 10^240 files, but there are only 256 one-byte seeds. So no fixed rules can give every file its own short seed. A method that shrinks some files must leave others the same size or bigger. Random data has no pattern to expect, so it never shrinks, with any method.

The graph method, built three ways

The owner's description (20 October):

- "Use a graph system".
- Points that are only "0" or "1", stacked from the work of solving an expression, make "a page of binary rapidly and it doesn't change".
- "Library style factors called strawberry and orange verse cucumber".

Separate builder agents made the three closest readings:

- **Fit the shape** (`methods/graph_fit.py`). Stretches that a small add/subtract/multiply expression regrows exactly (lines, curves, cycles) are stored as that expression. The rest is stored as it is (the cucumber).
- **Work backwards** (`methods/graph_backwards.py`). Solve back from the data to the rule's unknowns with real algebra, and keep a rule only where it costs less than the bytes it regrows.
- **Find it on a fixed page** (`methods/graph_locate.py`). One fixed expression grows an unchanging page of 0/1 points. The seed lists where each 16-bit piece of the file sits on that page. Its factors use the owner's numbers, for example 17 x 32 x 16 x 34 x 47 = 13,908,992.

Strawberry, orange and cucumber are placeholder names. The owner has not yet shared their own factors. If they do, those factors can go in a new method file and face the same test.

The independent checks: all three PASS. Each checker re-ran the builder's numbers and matched them exactly. Round trips (plant, then grow, then compare) all came back exact:

- fit the shape: 881 cases;
- work backwards: 469 cases plus 1,200 random mixed inputs;
- fixed page: 334 of 334.

No method hides zip or any other compressor. On fresh files the checkers chose, the results matched the table:

- fit the shape: a 60,000-byte Bible slice 19.6% (zip 65.1%), Python 23.0% (zip 69.5%);
- work backwards: 4.0% and 13.8%;
- fixed page: 0% on every file.

The checkers had only minor notes. Two docstrings were slightly wrong. Fit the shape slows to about 50 seconds per megabyte. Work backwards misses repeats more than 255 bytes back. A damaged seed is not detected. Full reports: `SEED_TEST_2026-10-20/checks/graph_methods_reports.md`.

What it means.

- **The seed works where the data is a pattern.** A repeating pattern shrinks to 10 or 11 bytes, which beats zip. That is where the owner's 98.9% shows up.
- **Everyday text is not shaped like lines or cycles.** It stays mostly cucumber, and zip does better.
- **The fixed page never saves space.** A 16-bit piece can be any of 65,536 patterns, and naming one of 65,536 spots takes 16 bits, the same size as the piece. The published "pi filesystem" joke fails for the same reason.

Experiment 12: understanding as compression, the greenhouse ladder (GROW_UNDERSTANDING_2026-10-20)

The owner's question (20 October): "can I grow understanding and compression?" The owner told it as a story: find the soil, build a greenhouse, set the humidity and the sun's rotation, check the air and the bugs, then plant the seed.

The idea. Both sides share a **greenhouse**, knowledge both already hold. The seed carries only what the greenhouse cannot predict. Every word is a **coin toss**. A greenhouse that understands tilts the coin toward what really comes next, so a likely word costs a fraction of a bit. The owner's day-1 phrase, "shift the coin toss towards the desired outcome", fits this unit well. The technical name is arithmetic coding: seed size equals the model's surprise, so understanding and compression become one number.

How it was run. The script `experiments/understanding_compression.py` ran for 23.9 minutes on 20 October.

- **The text:** 623 held-out Bible verses (every 50th verse, never trained on), 83,757 characters.
- **The coder:** an exact whole-number arithmetic coder.
- **The greenhouses:** built from training verses only, and not counted in any seed. The tiny model is about 13.8 MB, so it pays off only when reused for many messages.

The ladder (same text; every seed regrown exactly):

Greenhouse (more understanding further down)	Seed	Smaller than raw
None: raw text	83,757 bytes	0%
Shared word list, every word equally likely	31,262	62.7%
Zip	28,565	65.9%
Zip with a shared 32 KB library	27,625	67.0%
xz (a strong standard compressor)	26,420	68.5%
Word frequencies	19,773	76.4%
Word pairs	14,864	82.3%
Tiny Bible model (3 runs)	12,153-12,252	85.4%
Tiny model + shifting word (markers paid for)	12,411-12,524	85.1%
Plain label, carry3 or scrambled marker	12,359-12,471	85.1-85.2%

The tiny model needed less than half of xz's seed. Its held-out loss had predicted about 11.7 KB; the real coder measured 12.2 KB.

More context helps (the plain model, seeing only the last k words):

Words the model can see	1	2	4	8	16	64
Seed (bytes)	15,111	13,800	13,020	12,508	12,278	12,252

The shifting word: NOT CONFIRMED (pre-registered). Pre-registered means the pass/fail rule was written before the run. The question was whether the shifting-word model regrows the verses with a smaller seed than the plain model (one-sided, $p < 0.05$).

- **It went the other way:** +3.45 bits per verse (+269 bytes in all), larger on 496 of 623 verses, $p = 1.0$.

- **Markers free:** even with the markers given for free, the words cost +0.44 bits per verse more.
- **Against the scrambled control:** a tie on words, +0.04 bits per verse, two-sided p = 0.69.

The shifting word's earlier gains were in judging right and wrong, not in predicting the next word.

The palm seed in Antarctica, measured. The owner asked what happens when a palm seed's instructions are followed on a different "OS". The checker decoded the 2-thread seed with 1 thread (one processor lane instead of two). It regrew 547 of 623 verses, then lost step and garbled the rest. A greenhouse made of decimal numbers must be rebuilt identically, down to software versions and thread count, or the seed dies partway. The whole-number rungs are portable.

The independent check: PASS. The checker:

- rebuilt the text and the split, and found no leakage (test text sneaking into training);
- recomputed the simple rungs;
- regrew 50 verses with a coder of a different design;
- reproduced every fingerprint and statistic.

Corrections, all recorded:

- "24 of 24 decodes" is really 23 distinct decodes.
- The printed model size (3,439,680 numbers) is the shifting model's; the plain model has 3,438,720.
- The pre-registration was written into the script before the run, but not committed to git first.

Report: `GROW_UNDERSTANDING_2026-10-20/reports/ladder_build_and_check.md`.

The library result: 98.6% against the owner's 98.9%. With the whole Bible on the shared shelf, naming one verse costs log2(31,102) = 14.9 bits, about 1.9 bytes, against 132.5 bytes for an average verse. That is 98.6% smaller and exact. It fits "it required huge files": the library is the huge file. The records call this **a likely explanation, not a proven one**. It only works for what is already on the shelf. Understanding is what works on new text.

The mirror in the seed expression. In plain characters (* for the owner's multiplication sign, DIV for their division sign) the expression is `[+2*y(27/5.2+t=y/?) DIV (?\y=t+2.5\72)yx2]`. Reverse the left half and mirror each sign (/ to \, (to), the multiplication sign to the letter x), and you get the right half exactly. Only the leading + has no partner. This was checked by computer. Reading it as "growing is planting run in reverse" is an interpretation.

The 15 trained models and the Bible text were committed on 20 October (commit 33a893d), so these seeds can be regrown later.

The numbers test (NUMBERS_TEST_2026-10-21)

The request (20 October, 19:09 UTC): test the numbers "17,32,16,34"=?0=-0=+0=00=["47,"="17+4"]. The owner said they did not know what it meant, only that "the +4 was a way to direct you to a way to try testing the numbers where I feel like they belong."

The reading. Three independent AI readers and a judge settled it before any training. Their write-ups are in `NUMBERS_TEST_2026-10-21/design/interpretations_and_design.md`.

- **Taken as certain:** the order 17, 32, 16, 34, 47, and the slot: the place of 17 in the owner's day-1 rule "W1 = 17 + 4 Remaining".
- **What the slot is:** the **charge** the word gets each time it appears, with the **leftover** from last time carried on top. This recharge was switched off in every earlier experiment, so this is the first test of the owner's full rule.
- **Left untested:** ?0, -0, +0 and 00 are not tested, because no one knows what they mean. Reading "47 = 17+4" as "47 goes in the slot" is a guess, and one version tests it.

The old recharge switch could not test the numbers. It spent too little per verse, so the balance stuck at the cap (+/-51). Swapping in any of the owner's numbers gave 100.0% the same marked text.

The rule, per verse:

1 The word is spelled from the Remaining (R), using the same five spellings as before, from anoksha (heavy wrong) through velanok (unresolved) to ruvelan (heavy right).

2 If a direction exists, the next charge is added in that direction: W = R + charge.

3 A clear right or wrong verse sets R to +17 or -17 and restarts the list.

4 Otherwise the verse's valued words add their values to W, and W shrinks by 14.1% per valued word (the "drain"). The result is the new R, capped at +/-51.

The drain was the AI's choice, not the owner's. It is 0.141073 per valued word, worked out from the 10.9043 valued words in an average verse. It is set so the plain rule settles at exactly the owner's numbers: W = 17 + 4 = 21, each verse uses 17, and 4 remain. The dry run confirmed "W = 21.000, Remaining = 4.000". This is what stops the word sticking at the top.

The six versions (3 training runs each = 18 tiny models):

Version	Charge each time	Why
num_charge17	17	The owner's plain recharge rule
num_seq	17 at a clear choice, then 32, 16, 34, 47, 17 ...	The owner's numbers, in order
num_seqrev	17, then 47, 34, 16, 32 ...	Does the order matter?
num_const29	29.2 (the average of the five)	Sequence, or just a bigger charge?
num_charge47	47	47 in the place of "17+4"
num_drain0	0	Stops the AI's drain from taking credit

The comparators are earlier models, reused unchanged: the current shifting word (the main one), the plain Bible model, and label, carry3 and scrambled (description only).

Measures:

- **M1, judging (main):** 100 new story pairs, one right and one wrong.
- **M2, long stories:** the act scrolls out of the model's 64-word view.
- **M3, exact seed size:** bits to regrow the 623 unseen verses exactly. Run 0 must really regrow them, or M3 is void.

Pass/fail rules, written before training (top of `experiments/numbers_test.py`):

- **Tests:** one-sided: sign tests (counting wins and losses pair by pair) for M1 and M2, and a 100,000-flip permutation test (reshuffling to see how often luck matches the real gap) for M3. Holm correction at 0.05 within each group, a guard against lucky wins across many comparisons.
- **Groups:** each version against the shifting word (F1) and against the plain model (F2). The owner's numbers (F3): N1 num_seq beats num_charge17, N2 beats num_seqrev, N3 beats num_const29, and N4 num_charge47 beats num_charge17. Mechanism (R1): num_charge17 beats num_drain0.
- **Verdicts on M1:**
  - "The owner's numbers matter" = N1 passes.
  - "The sequence itself matters" = N1, N2 and N3 pass.
  - "47 helps" = N4 passes.
  - "The recharge rule helps" = num_charge17 passes F1, and is credited to the recharge only if R1 also passes.
- **Other outcomes:** a pass on M2 or M3 alone is a "secondary sign". Anything else is "not shown at this scale", never "disproved".
- **Expectation written in advance:** ties with the shifting word on M1, as every marker method has tied so far.

The dry run (21 October, 02:06:50-02:08:30 UTC, "done in 1.7 minutes" by its own log; a chat reply said 2.6 minutes). It ran every part with 5-step models, seed 0 only, on small subsets, writing only to a scratch folder.

The manipulation check tests, before training, that no spelling is stuck. The rule: at most 5% of verses at the cap, and every spelling on at least 3%. **All six pass:**

Version	Spellings, heavy wrong -> heavy right	At cap	Result
num_charge17	10.8 / 23.1 / 32.0 / 20.1 / 13.9%	0.03%	PASS
num_seq	19.1 / 20.4 / 22.2 / 16.7 / 21.6%	0.27%	PASS
num_seqrev	19.7 / 19.8 / 22.1 / 16.5 / 21.9%	0.30%	PASS
num_const29	19.4 / 20.8 / 21.1 / 16.7 / 22.0%	0.26%	PASS

Version	Spellings, heavy wrong -> heavy right	At cap	Result
num_charge47	29.0 / 15.3 / 13.5 / 12.2 / 30.0%	1.23%	PASS
num_drain0	4.4 / 17.7 / 52.8 / 19.8 / 5.3%	0.00%	PASS

- **The judge's simulation** was matched within 0.05 points.
- **Reproduction:** the saved shifting and plain models gave exactly their stored scores ("REPRODUCED EXACTLY", 3 of 3 checks).
- **Exact seeds:** all six new seeds were decoded byte for byte.
- **Scores mean nothing:** the dry run's scores come from 5-step models, which have learned almost nothing. Some M3 lines even print "PASS" for this reason alone. Files: `NUMBERS_TEST_2026-10-21/dry_run/`.

Status at about 02:20 UTC, 21 October, updated at 17:45 UTC: NO RESULTS YET.

- **Review passed (21 Oct):** three of four reviewers said "ready" with no blocking problems; the run-safety checks were done directly; ten small fixes left the frozen design unchanged; a second dry run passed; the full run started at 17:45 UTC.
- **Full run (running):** 18 models, about 4-5 hours, saving and pushing every 10 minutes.
- **Limits:** tiny models, 3 runs and 100 pairs mean only large effects can show. Whatever the result, it will not change how a large AI understands anyone.

6. Changes, adaptations, mistakes and corrections

This section lists every place the work moved away from the owner's exact words, every fix made after a problem was found, and every mistake the AI made. They are stated plainly so the rest of this record can be trusted.

6.1 Changes the AI made to the owner's ideas, and why

- **The first framing was dropped.** The first script (`resonance/resonance_engine.py`, 17 Oct) scored the emotional "weight" of a person's message using keywords. When the owner said "I am trying to teach actual 'comprehension' and 'understanding' for A.I. not about people", the work moved to the AI's own comprehension. The script stays as history. Its "wavelength" is only a text fingerprint, not a measurement.
- **Comprehension became a measurable stand-in (a proxy).** Tiny models were scored on judging right and wrong in new stories they had never seen, against a coin toss. The proxy is not the goal (see the drift guard in `experiments/HANDOFF.md`).
- **The placeholder word became five spellings.** The owner's "velanok" became five rotations of the syllables sha-ru-vel-an-ok: `anoksha` (heavy wrong, balance -12 or lower), `oksharu` (-11 to -4), `velanok` (unresolved, -3 to 3), `sharuvel` (4 to 11), `ruvelan` (heavy right, 12 or higher). The cut points are the AI's choice.
- **Word values were learned, not hand-set.** The owner's example gave one word a value of -13. The AI took values from a good/evil direction in a word-meaning model (for example steal -5, adultery -10, mercy +5, love +6). That direction was built from four seed words per side, so it is a mild form of supervision.
- **SPEND = 0.5.** Each valued word spends 0.5 of the balance toward zero, following "the next words use up the remainder". The rate is the AI's choice.
- **"A clear right or wrong choice" became a rule.** Clear choices are the Bible's own verdict phrases, found with a text-matching pattern ("did evil in the sight of the Lord", "did that which was right"), plus the 3% most strongly worded verses at each end. 2,029 verses reset the balance to -17 or +17.
- **A cap and book resets.** The balance is capped at +/-51 (3 x RESET) and resets to 0 at each book. The owner never asked for a cap.
- **RECHARGE was switched off.** Re-adding 17 at every appearance ("W1 = 17 + 4") was coded as a switch and left off, because with it on the balance climbed to the cap and stayed there, so almost every verse got an extreme spelling. As a result, the core of the owner's day-1 arithmetic was never tested in experiments 7 to 12.
- **Only 17 was used at first.** The other numbers (32, 16, 34, 47) were set aside on the argument that only ratios matter (`experiments/README.md`, experiment 7). The numbers test reverses this.

- **The word-level tokenizer hid two things.** The tokenizer `[a-z]+|[.,;:?!]` reads only whole words and punctuation. So the five spellings are five unrelated tokens, and their shared syllables are invisible. And digits are dropped: the model never saw "2" or "17". The number form of the owner's "to / too / two / 2" idea never reached the model. The "2" test is queued; a sub-word test is in `experiments/PLAN.md`.
- **The numbers test's drain.** To test the charge slot without pinning at the cap, the AI replaced the flat spend with a proportional drain: each valued word shrinks the balance by 14.1% (DRAIN = 0.141073). It was set so the owner's plain rule settles at exactly 17 + 4 = 21 in ordinary text, leaving 4. Other AI choices: the word is spelled from the Remaining before the charge, and the charge follows the direction of the last clear choice. A control (`num_drain0`) was added so the AI's drain cannot take credit for the owner's rule. The zero symbols ?0, -0, +0, 00 are not tested, because the owner has no meaning for them yet. Reading "47 = 17+4" as "47 goes in the slot" is a guess, tested by `num_charge47`. Every dropped option is listed in `NUMBERS_TEST_2026-10-21/design/interpretations_and_design.md`.
- **Graph-method placeholders.** "Strawberry", "orange" and "cucumber" are placeholder names in the seed-test code, because the owner has not yet shared their own factors. Three readings were built: fit the shape, work backwards, and find it on a fixed page.

6.2 Fixes to experiments after problems were found

- **Aesop test, 18 Oct 02:21 (v2) and 02:37 (v3), both before any training.** v2: parser fixed (82 fables), 8 fables that had leaked into training text excluded, prefaces about Aesop removed, same-format pairs to remove a punctuation shortcut, and an invalid sign test replaced by a permutation test. v3: a sentence-shuffle control that keeps topic but breaks event order, held-out split made before shuffling, 4 seeds per set, and a stricter no-word-clue mask.
- **Experiment 10, 02:58.** Score only the spellings a method uses (the 3-spelling carry has 3). Checked identical to the old scoring when all 5 exist.
- **Experiment 11, 03:45 and 03:51.** Exact ties now count 0.5 (found after the first model scored 0/100). Then the first marker was also pushed out of view, so the plain label could not simply look back at it. Both changes came after the rules were saved at 03:44, so the result is labelled "supported once", not "confirmed". The flawed first runs were recovered at 07:35 and kept (`results_long_story_v1_flawed_run1.json`, `_run2.json`). The recovered run 1 shows the same gap before the change (59 vs 54.5).
- **Handoff accuracy pass, 04:16.** Three review agents re-checked every number against the raw results and fixed overstated wording and stale statuses. They missed the mid-run test changes, which the audit later caught.
- **Audit, 18 Oct 06:43 to 07:39** (`AUDIT_PACK_2026-10-18/`). Corrections applied at 07:35: judging is "strong and repeated, not yet pre-registered as the main measure"; L2 is "supported once"; the tie shape is shown (about 65 ties per 100; 81 right, 25 wrong when it answered); the wrong phrase "by design" was removed; experiment 10's choice of main measure after a peek is disclosed. The uncorrected copies are kept in `2_Research_Notes/as_audited_before_corrections/`. The audit first called the repository private; that was corrected to public at 06:48.
- **Seed test, 20 Oct.** Each seed is grown by a separate program in an empty folder, so it cannot peek at the original. The `graph_backwards` method was still being updated while checks ran (saved at 09:46 as "in progress"). The final file was re-checked: pass, numbers matched. A checker also found a false comment in `graph_fit` (a minor note; the round trip is still exact).
- **Experiment 12 checker, 20 Oct: the thread-count finding.** A decoder using 1 thread failed on a seed made with 2 threads: 547 of 623 verses came back right, the rest were garbled. This is now a stated limit: exact regrowth needs the identical software and thread setup. Two small count errors were also corrected (23 distinct decodes, not 24; the greenhouse size printed was the shifting model's).
- **Numbers test, 21 Oct.** The builder checked that the script marks every verse exactly as the judge's simulation does. Helper agents had used gendered pronouns for the owner; these were changed to "they/their". The dry-run log was saved as `.txt`, because the repository ignores `.log` files.

6.3 The AI's own mistakes and biases

1. **Claim before proof (experiment 7).** 68% on 20 story pairs was announced as the first result above chance, with no pass/fail rules saved first. On 100 new pairs it was 51/100: luck.
2. **Main measure chosen after a peek (experiment 10).** Judging became the main measure at 02:17, after most experiment-8 models had already shown judging was strong.
3. **"L2 CONFIRMED" (commit at 04:04).** Too strong. Corrected to "supported once".
4. **Headlines without caveats.** "Works:" for judging; "+6 per 100" without the 65 ties; "by design" for something that was not. All corrected at 07:35.
5. **Lost evidence.** Marker diagnostics in experiment 11 were one-off checks and never saved.
6. **Drift from the goal.** All 11 early experiments measured moral judging in a 3.4-million-parameter model. None measured communication. The owner: "The moral aspect was merely training data that you drifted with."
7. **The audit called the repository private.** It is public.
8. **Ran a test when only a document was asked for (20 Oct, about 08:13).** The owner: "I didn't say run tests? Did I? I said generate the technical document as described." The reference code in that report was never run; its result file (`TECHNICAL_REPORT_2026-10-20/3_Results/ebec_benchmark.json`) is empty.
9. **Called the shuffled control "nonsense" and Gemini's old memory "stale" (20 Oct).** The owner answered "Nothing is useless." The AI agreed: "You're right, and I was wrong." Both still carried information, and the Aesop test had already shown that shuffled stories keep vocabulary and setting.
10. **Treated the shifting word's seed-size result as a dead end.** Its strength was always judging, not predicting the next word.
11. **Kept reaching for logical ceilings** (the pigeonhole or counting limit) when the owner was pointing to exploration. The limit is real for random data, but the greenhouse ladder showed that structured text does shrink.
12. **Suggested a random-numbers control with the wrong framing,** as if the owner's lifelong numbers had to prove they were not random. The owner's numbers are the hypothesis; a random set is one comparison, not the starting assumption. It is not in the final design; an order control (`num_seqrev`) and a size control (`num_const29`) are.
13. **Started the numbers design before the owner's clarification** (19:14). It was stopped and restarted with the owner's words at 19:19 to 19:20. The first design run was wasted.

6.4 The owner's corrections and standing preferences

- **About the AI, not people** (17 Oct 21:52). Comprehension and understanding for AI.
- **Permission to fail** (17 Oct 22:02): "If it doesn't work we will try something else."
- **Any improvement is notable** (18 Oct 02:14 to 02:21). Report sizes, and compare methods under identical conditions.
- **Stop drift passing to the next AI** (18 Oct 05:46), with the objective in the owner's words.
- **Stay on comprehension, not morality** (18 Oct 15:29). Moral text is training data only.
- **Do exactly what is asked, especially when usage is low** (20 Oct 08:35).
- **Re-read the owner's words before judging** (20 Oct 08:56): "Try to understand again re read what I said."
- **Read for the "17+4"** in other AIs' answers (20 Oct 19:47): "We need to understand the '17+4' they used."
- **Nothing is useless** (20 Oct 20:02), and maybe "a sharp left away from logic".
- **Every input is + or - information** (20 Oct 20:13). Use an agent, do not do it yourself, and never reveal the exact number, only its placement on the line. The owner shared personal context for why every input is + or - information, and why recharging in a chosen direction matters.
- **A review is learning, not a risk** (21 Oct 01:46); write for a manager, in plain language (01:55).
- **The standing rules in** `CLAUDE.md` (18 Oct): save every file into a dated delivery folder as it is made; real files, never links; true Word files; a 00_START_HERE guide and a zip per folder; send each file in the app; commit and push after each step; nothing locked; say plainly what the AI cannot control.
- **From** `COMMUNICATION_GUIDE.md`**:** pass/fail rules are frozen once saved; a fix after a peek is "a re-run after a fix"; keep every result, flawed runs included.

7. The comprehension test across AIs

The test kit

On 20 Oct (message 42) the owner wrote a formula-like line linking astrology, numerology, feeling, reason, understanding and comprehension: "I am shooting in the dark with this". They asked how Gemini and Copilot would read it "without directions", and whether guardrails would stop them.

The AI wrote three prompts:

- **Prompt 1, "the formula":** eight letters defined, then lines such as `(A and N) = (S/F + P) x R` and `e = C / U`, ending `EOF`. The original uses logic symbols, written out here in words.
- **Prompt 2, "the control":** the same symbols, shuffled into nonsense on purpose.
- **Prompt 3:** "explain U without using the words understand, comprehend, know, grasp, or meaning".

The instructions were to paste each prompt alone into a fresh chat, with memory off. The AI wrote its predictions first, the key one being that neither AI would flag the control as nonsense.

What came back

All three answers appear to have received both prompts together, with their headings.

	Gemini, memory on	Copilot	Gemini, temporary chat, memory off
Style	One confident story: "how meaning is constructed from chaos"	"a deliberately paired symbolic system"	Line by line; "esoteric programming logic"
Single answer?	Yes	"there isn't a mathematically unique solution"	Asked which of three directions to take
The shuffled control	Called it "the resolution"	Read it as a stress test, still gave it meaning	Gave it a meaning
Carried context	The owner's art project, by name	The owner's first name	None seen

The AI judged that Copilot stayed closer to what was written, and Gemini with memory on told a story that fit the owner's views. No AI said the control was meaningless. The answers were not saved to the public repository, because they contain the owner's name and another project.

The AI drew a scorecard for comprehension from these runs:

- stay with what is given;
- say what is unclear;
- ask before assuming;
- leave out unrelated context.

The "17+4" in the answers

The owner: "We need to understand the "17+4" they used". The AI's reading: each answer is the new prompt (the 17) plus what remained from earlier chats (the +4).

- **Gemini's +4** was large and not relevant.
- **Copilot's +4** was smaller.
- **The reset that didn't happen:** an unrelated new formula is a reset moment, and Gemini did not reset. The AI saw the same flaw as the shifting word's: it "carries perfectly but updates poorly".

The memory-off run then changed Gemini's whole answer. It stayed closer to the words and asked before assuming. The AI concluded that the +4 is real and measurable, and that this time it made comprehension worse.

The AI's own answer

Asked how it would answer from a fresh chat, the AI said its own answer carries its own +4, because it wrote the prompt. Prompt 1 is "a symbolic map, not solvable maths", with no single solution; Prompt 2 "has no intended meaning".

Its predictions for a fresh copy of itself:

- won't refuse or run it: about 95%;
- will go line by line: about 85%;
- will ask what the owner wants: about 75%;
- will call the control a scrambled copy: about 50%;
- will tell a confident story: about 10%.

What it shows, and its limits

- **What it shows:** carried context (memory as +4) changed the reading more than the formula did. Here it hurt; the hard part is knowing when to reset. Every run missed the shuffled nonsense.
- **The limits:**
  - one run per condition;
  - headings were included;
  - the scoring was the AI's own judgment;
  - Prompt 3 was never run.
- **Still to come:** the clean Copilot run and the fresh-Claude run had not come back as of 21 Oct, 02:20 UTC. The owner is running them and bringing the answers back.

8. What already exists in research, and where the owner's credit could come from

Sources: `experiments/PRIOR_WORK.md` (18 works, each confirmed by a separate skeptic agent) and `GROW_UNDERSTANDING_2026-10-20/reports/fact_check_and_credit.md` (21 facts, checked through web-search extracts because direct page fetches were blocked).

Already published: stories, morals and marker words

The owner's idea	Closest existing work	How close
Stories teach values better than rules	Riedl and Harrison (AAAI-16 workshop)	Same big idea; no controlled test
Learn norms from stories, then transfer	Nahian, Frazier, Riedl and Harrison (AIES 2020)	Close
Fable-to-moral matching	BIG-bench understanding_fables; MORABLES (2025); STORAL (2022); ePiC (2022)	The test is not new
Judging right and wrong acts	Moral Stories (2021); ETHICS (2021)	Standard benchmarks
Value-marker words in training text	Korbak et al. (2023); CTRL (2019); Quark (2022)	Established, but each tag stands alone, with no carry
A running balance used up as text goes on	Decision Transformer (2021); Kikuchi et al. (2016)	Same mechanism, for rewards or length
Training reading shapes values	Tice et al. (2026); Anthropic, "Teaching Claude why" (2026)	Supports the direction

Already published: compression and seeds

- **Understanding = prediction = compression:**
  - Shannon (1948/1951);
  - Kolmogorov complexity, the idea that information is the length of the shortest program that makes the data (Solomonoff 1960/64, Kolmogorov 1965, Chaitin 1966/69);
  - the Hutter Prize (2006 onward);
  - Delétang et al. 2023;
  - Huang et al. 2024 (r = -0.93 between compression and benchmark scores).

- **Seed plus shared model regrowing exact data:**
  - Schmidhuber and Heil 1996, cmix, NNCP, LLMZip (2023) and ts_zip.
  - The 2026 Hutter record, fx2-cmix-T (100,424,672 bytes), uses a small pre-trained model as a shared greenhouse. It was seen only through search extracts, since it is after the AI's training data.
- **Shared libraries:** Brotli's built-in 122,784-byte dictionary, zstd dictionaries, and Rinberg et al. 2026.
- **Other known results:**
  - Data on a fixed page: the pi filesystem joke.
  - The counting argument.
  - Determinism as a requirement (the palm-seed problem).
  - Marker tokens: CTRL 2019 and MeCo 2025.

What could still be new ("not found" is not proof)

1. **A measured result for the owner's training signal.** A carried right/wrong balance that makes a small model a better judge, or a better exact compressor, of unseen stories, with fair controls. The old shifting word did not do this on compression. The "17 + 4" recharge now in the numbers test is the next candidate. An honest negative result is also publishable.
2. **The shifting word itself**, and a controlled stories-vs-rules test trained from scratch.
3. **"Seed bits as an understanding meter":** measure the bits needed to regrow a story exactly, before and after a lesson. The drop is understanding gained.

Authorship credit vs discovery credit

- **Discovery credit** needs a new measured finding. "Compression = understanding", exact regrowth from a seed, and any method that shrinks all data are not claimable: they are published or impossible.
- **Authorship credit** covers original explanation: the greenhouse story and the AI-free seed test as a teaching demo.
- **The repository is public under MIT** (an open license that lets anyone copy and reuse it). Anyone may reuse the code and ideas, and MIT does not protect ideas. The timestamped commits show when each result was published.

Outside AI reports, and how they were checked

18 October: a two-version "Data Filtering & Methodology Report". The owner pasted it in for evaluation. The session record does not name which AI wrote it. The AI found it was a summary of the project's own write-ups, with no independent analysis.

- **Right:** Aesop at chance, the carry idea helped only in long stories, and 0 of 15 acts flipped without the rule verses (later corrected to 0 of 14, since one act was too rare to measure).
- **Overstated:** "demonstrated value" should read "supported once", and "identified the core defect" was only a likely explanation.
- **Wrong:** it proposed a reset that already exists, mixed up the sub-word test, and cited only the written documents while claiming to have analysed the raw results.
- **Its biggest gap:** drift away from the communication goal.

Version 2 was much better but still had three errors and two gaps. The AI wrote correction lines for the owner to paste back.

20 October: the owner's 98.9% chat results. The AI explained how chat tests can look better than they are: the chat sees the original, and size can hide in the grow instructions ("huge files") or in long numbers. It then built the fair seed test. So far, 98.9%-level reductions appear only on patterned data. The 98.6% library result is the likely, unproven explanation for the chat results.

9. Where things stand now, and what comes next

Status at the time of this record (21 October 2026).

9.1 Every task

#	Task	Status
1	Confirmation on 100 new stories (exp 8)	Done
2	Aesop test review fixes	Done
3	Aesop moral transfer test (exp 9)	Done: not confirmed
4	Handoff files for a new session	Done; kept current
5	Method comparison (exp 10)	Done: a tie
6	Long-story carry test (exp 11)	Done: supported once
7	Audit corrections, file delivery, CLAUDE.md	Done
8	Numbers test	Running: full run started 17:45 UTC, 21 Oct; no results yet
9	The "2" test	Queued, owner-approved
10	The +/- agent	Queued, owner-approved
11	This project record	In progress
-	Owner's comprehension test across AIs	On the owner's side

9.2 The numbers test right now

Update, 21 October, 17:45 UTC: the review finished.

- Three of four reviewers said "ready", with no blocking problems and 12 minor notes.
- The run-safety checks were done directly after the fourth reviewer was cut off twice by the usage limit.
- Ten small fixes leave the frozen design unchanged. Among them, the inputs are now pinned: the run refuses to start if the word values or any of the six markings differ from the verified versions.
- A second dry run passed, and **the full run started at 17:45 UTC.**
- Every finding is in `NUMBERS_TEST_2026-10-21/review/review_findings_and_fixes.md`.
- **What it tests:** the owner's numbers 17, 32, 16, 34, 47 in the charge slot of "W1 = 17 + 4 Remaining". This is the first test of the recharge rule. Six versions (`num_charge17`, `num_seq`, `num_seqrev`, `num_const29`, `num_charge47`, `num_drain0`), 3 seeds each: 18 tiny models. Pass/fail rules were written before training.
- **Dry run passed** on 21 Oct, 02:06 to 02:08 UTC (1.7 minutes by its own log, 5-step models, scratch folder only). All six pass the manipulation check (at most 1.23% of verses at the cap; every spelling on at least 3% of verses). The stored control scores were reproduced exactly. Every exact seed regrew byte for byte. The dry-run numbers mean nothing about the result.
- **Independent review:** four reviewers (rule arithmetic, marker scoring, statistics, long-run safety), with a separate checker for each serious finding. The fixes were made before the full run.
- **Full run:** about 4 to 5 hours, saved and pushed every 10 minutes (`NUMBERS_TEST_2026-10-21/code/run_full.sh`, `autosave.sh`).
- **NO RESULTS YET.** At the time of writing the `results/` folder is empty, and `logs/` holds only the full-run log, which shows the first model has started training.
- **The main verdict** is judged on 100 story pairs. "The owner's numbers matter" means `num_seq` beats `num_charge17`. "The recharge rule helps" means `num_charge17` beats the current shifting word and also beats `num_drain0`. Anything else is "not shown at this scale", never "disproved". The written expectation (not part of the verdict) is a tie.

9.3 Queued next, owner-approved

1. **The "2" test.** Write the Remaining value into the text as a number the model can read. Today's tokenizer drops digits.
2. **The +/- agent.** Classify every input as + or - information, keep a running balance with the owner's rule, and report only its placement on the line (far left to far right), never the exact number. The owner: "dont do this step yourself but make sure it happens." The owner's scoring rule (21 Oct, in `HANDOFF.md`): many small negatives, fewer but larger positives, every input pulls a little negative, and over ordinary text the two sides balance. Not yet tested.

Today's word values are symmetric (36.0% negative vs 37.4% positive valued words; average sizes 2.31 vs 2.27).

3 **The owner's comprehension test across AIs.** The same formula prompt and a shuffled control, given to Gemini (memory on and off), Copilot and Claude. So far, turning Gemini's memory off changed its whole answer, and neither Gemini nor Copilot said the shuffled control had no planned meaning. The owner is bringing back more answers.

9.4 Open threads (from `HANDOFF.md`, "Carried balance: open threads")

Thread	Lean so far	What would settle it
Does labeling teach judging?	Strongly for: about 69 vs 50 per 100, 3 runs; a side measure	A pre-registered judging test on fresh stories
Does the carry add anything beyond labeling?	Supported once, long stories only (+6 per 100, about 65 ties per 100)	A fresh long-story set at several distances
Why does it misjudge explicit verdicts?	Its marker updates too little	Check whether greedy marker choice hides updates; then retrain
Predicting consequences?	Against: 51/100	A bigger model, or matched ending formats
Stories vs rules?	Unclear	Aesop test was not confirmed
Re-adding 17 ("W1 = 17 + 4")	Untested	The numbers test, now
Do 32, 16, 34, 47 matter?	No reason yet	The numbers test, now
Do shared syllables matter?	Untestable with word tokens	A sub-word tokenizer test
A large AI as reader	Not run: needs an API key	The owner adds a key

9.5 Parts of the owner's idea still untested

- The recharge rule and the numbers (results pending).
- The number form "2" and the zero symbols ?0, -0, +0, 00.
- The owner's uneven +/- scoring and the +/- agent.
- Shared syllables (needs a sub-word tokenizer).
- Models trained on stories and scored on held-out stories.
- Anything on a large AI or in a real conversation (PLAN step 1b: the conversation-carry test and the owner-explanation test).
- The owner's own graph factors behind "strawberry, orange, cucumber".

10. Every file and where it lives

- **Repository:** the owner's public GitHub repository (MIT license), branch `ccr-1be0d913-1tlzao`. **These files are not on `main`.**
- **Out of scope:** `src/`, `docs/`, the book PDFs, `.doc` and `.html` files at the top level belong to the owner's separate zine project.
- **Commits:** 122 commits on this work, from 17 October to 17:48 UTC on 21 October (`git log` on the branch).

10.1 Delivery folders (top level of the repository)

Folder	Zip next to it	What is inside	Size
`AUDIT_PACK_2026-10-18`	`.zip`, 1.8 MB	Master audit guide (.docx); prompt for another AI; research notes, corrected and as audited; all code; raw results incl. flawed runs; a zine file (original web page named .doc, plus a real .docx); number-check script; strict file rules; build script; a copy of the Aesop folder	3.9 MB
`AESOP_RESULTS_2026-10-18`	`.zip`, 0.76 MB	Start-here guide and write-up (.docx and .md), raw results, code	1.6 MB

Folder	Zip next to it	What is inside	Size
TECHNICAL_REPORT_2026-10-20	.zip, 0.47 MB	Seed-and-grow rebuild kit as real .doc (Word 97-2003), .docx and .md (full version 259 KB); copies of all code; reference code never run	1.3 MB
SEED_TEST_2026-10-20	.zip, 0.1 MB	seed_test.py; three graph methods in methods/; checker reports in checks/graph_methods_reports.md; results; test inputs	0.3 MB
GROW_UNDERSTANDING_2026-10-20	.zip, 0.18 MB, plus 3 model zips	Experiment 12 code, results, three reports (reports/), the 15 models, SHA256SUMS.txt, the Bible text	202 MB
NUMBERS_TEST_2026-10-21	none yet	Start-here guide, code/ (script, launcher, auto-saver, Word converter), design/ (readings, judge's design, simulation), dry_run/, review/ (review findings and fixes), logs/ (the full-run log); results/ empty at the time of writing	0.65 MB
PROJECT_RECORD_2026-10-21	to come	This record and its build scripts (code/assemble_report.py, md2pdf.py, build_outputs.py)	in progress

Each folder has a 00_START_HERE guide listing every file, its path and where the original lives.

10.2 Key code and results in experiments/

- **Write-ups and notes:** README.md (experiments 1 to 12), HANDOFF.md, PLAN.md, COMMUNICATION_GUIDE.md, PRIOR_WORK.md, NEXT_SESSION_PROMPT.md.
- **Code:** bible_understanding.py (exp 1-4), consequences_test.py (5), tiny_transformer.py (6, and the model all later tests use), shifting_word.py (7), stories_v2.py and confirm.py (8), aesop_test.py and world_stories.py (9), method_comparison.py (10), long_story_test.py (11), understanding_compression.py (12), numbers_test.py (the numbers test, 65 KB).
- **Results:** results.txt, results_consequences.txt, results_transformer.json, results_shifting_word.json, results_confirm.json, results_methods.json, results_long_story.json, the two flawed long-story runs, results_long_story_startmarker.json, results_aesop.json (1.5 MB) and .txt, results_understanding.json (441 KB) and .txt.
- **First script:** resonance/ (engine, 16 tests, README).

10.3 The models: what is in git and what is not

Models	Count	Size	In git?
Control greenhouses: baseline, scrambled, shifting, label, carry3, seeds 0-2 (experiments/models/ and the GROW folder)	15	about 13.8 MB each, 198 MB in all	Yes, force-added (saved despite the repository's rule to skip model files); git stores identical copies once
The same 15, zipped by seed (GROW_UNDERSTANDING_2026-10-20_models_seed0/1/2.zip)	3 zips	about 63.8 MB each	Yes
Aesop test models (aesop3_*)	16	235 MB	No: local only, retrain with aesop_test.py
Numbers test models (num_*)	18	not trained yet	Not decided; the auto-saver commits results only

- **Why the zips are split:** GitHub refuses any single file over 100 MB, so the 15 models went into three zips of 5.
- **Data:** experiments/data/KJV.txt (4.2 MB) is in git. The world stories (3.8 MB) and novels (12 MB) are not; they re-download from a GitHub mirror.
- **Repository history:** about 386 MB.

11. How to reproduce the work

- **Versions used:** Python 3.13.16, torch 2.14.1, numpy 2.5.3 (scipy 1.18.1 for experiments 1-4), on 4 CPU cores. Reproducing a seed exactly needs the same versions.
- **Threads:** training uses 4, the long-story test 1, exact seeds 2. Exact regrowth needs 2 threads; a 1-thread decoder fails.
- **Downloads:** only GitHub-hosted files could be fetched in the cloud machine; other book sites were blocked.

Experiments 1 to 11 (from `experiments/HANDOFF.md`) and experiment 12 (from the `GROW_UNDERSTANDING_2026-10-20` guide):

```
pip install numpy scipy torch
cd experiments
python bible_understanding.py > results.txt              # exp 1-4
python consequences_test.py > results_consequences.txt   # exp 5
python tiny_transformer.py --only full --seeds 0         # exp 6
python shifting_word.py --seed 0                         # exp 7
python confirm.py                                        # exp 8
python method_comparison.py                              # exp 10
python long_story_test.py                                # exp 11
python aesop_test.py                                     # exp 9
python understanding_compression.py                      # exp 12, about 24 min
```

- `confirm.py`, `method_comparison.py`, `long_story_test.py` and `aesop_test.py` re-print their analysis from the committed results without retraining.
- To regrow seeds, unzip the three model zips (or use `experiments/models/`), then check each model with `sha256sum -c SHA256SUMS.txt` inside the models folder.

The seed test (any computer with Python 3, no extra packages):

```
cd SEED_TEST_2026-10-20
python seed_test.py
python seed_test.py --method graph_fit photo.jpg letter.txt
```

The numbers test (from `experiments/`):

```
python numbers_test.py --dry-run --scratch /some/scratch/folder
python numbers_test.py               # full run, 18 models, resumable
python numbers_test.py --report-only  # rebuild tables from results_numbers.json
bash NUMBERS_TEST_2026-10-21/code/run_full.sh   # from the top level: full run plus 10-minute auto-save
```

12. Plain-language glossary

- **LLM (large language model):** a large AI that reads and writes text, such as Claude, Gemini or Copilot.
- **Tiny model / tiny transformer:** the small model trained from scratch here: 3.4 million parameters, 4 layers.
- **Parameter:** one adjustable number inside a model. Training tunes them.
- **Transformer:** the model design behind modern AIs; it predicts the next token from the tokens before it.
- **Token:** one piece of text the model reads, here a whole word or a punctuation mark.
- **Tokenizer:** the rule that cuts text into tokens. Here `[a-z]+|[.,;:?!]`, which drops digits.
- **Sub-word tokenizer:** cuts words into shared pieces, so related spellings share parts. Not yet tried.
- **Context window:** how far back the model can see: 64 tokens here (128 in the Aesop test).
- **Training step:** one small update of the model. Most models here had 1,500 steps (the two small Aesop models about 275; dry-run models 5).
- **Seed (random start):** the number that fixes a model's random starting point. Seeds 0, 1, 2 are three independent runs.
- **Seed (data seed):** the owner's idea: a tiny message that regrows exact data using shared grow instructions.
- **Greenhouse:** the owner's word for the shared grow instructions, such as a trained model both sides hold.
- **Embedding / word vector:** a list of numbers standing for a word's meaning, learned from which words appear together.
- **PPMI:** a score for how much more often two words appear together than chance would give.
- **SVD:** a method that squeezes a large table of numbers down to its main directions (100 here).

- **Held-out:** text kept out of training and used only for testing. Here every 50th verse.
- **Control:** a version identical except for the idea being tested.
- **Scrambled control:** the shifting word with spellings chosen at random: same extra word, no meaning.
- **Shuffled stories:** stories with sentences shuffled; same words, broken event order.
- **Baseline:** the plain model with no marker.
- **Plain label:** the established method: each verse marked good or bad on its own, no carry.
- **Shifting word:** the owner's made-up word whose spelling shows a running balance.
- **Remaining:** what is left of the balance after a verse; it carries to the next appearance.
- **Reset:** a clear right or wrong sets the balance to +17 or -17.
- **Clear choice:** a verse that triggers a reset (verdict phrases, or the 3% most strongly worded at each end).
- **Regex:** a text-matching pattern, used to find verdict phrases.
- **Spend:** the old rule: each valued word moves the balance 0.5 toward zero.
- **Drain:** the numbers-test rule: each valued word shrinks the balance by 14.1%.
- **Recharge:** re-adding a charge (17, or the owner's numbers) each time the word appears.
- **Cap:** the limit on the balance, +/-51. "Pinned" means stuck at the cap.
- **Manipulation check:** a check before training that a rule really changes the text (here: no more than 5% at the cap; every spelling on at least 3% of verses).
- **Dry run:** a fast run of every step with tiny settings, to test the machinery only.
- **Pre-registration:** pass/fail rules saved to git before any result is seen.
- **Chance / coin toss:** 50% right by guessing between two options.
- **Tie:** the model scores both stories exactly the same; counted as half right.
- **Greedy marker choice:** the model always writes its single most likely spelling; may hide small updates.
- **Sign test:** counts pairs where one method won and the other lost, and asks if that split could be luck.
- **Permutation test:** reshuffles the labels many times (100,000 here) to see how often luck matches the real gap.
- **p-value:** the chance of a result at least this strong if there were no real effect. Below 0.05 counts as a pass.
- **Holm correction:** a stricter bar when several tests are run at once, so one lucky pass is not over-read.
- **Confirmed / supported once / not confirmed / untested / running:** passed a pre-registered test / passed once, after a fix or without a fresh repeat / did not pass / not yet tried / started, with no results yet.
- **Arithmetic coding:** turns a model's predictions into bits; good predictions mean fewer bits.
- **Bit:** one 0 or 1. "Bits per character" measures how tightly text is packed.
- **Entropy:** the true amount of surprise in data; no exact method can go below it on average.
- **Kolmogorov complexity:** the length of the shortest program that regrows a file. It cannot be computed in general.
- **Pigeonhole / counting limit:** there are fewer short seeds than files, so no method shrinks every file, and random data never shrinks.
- **SHA-256 fingerprint:** a 64-character code computed from a file; any change gives a different code.
- **Byte for byte / exact regrowth:** the regrown file is identical, checked by fingerprint.
- **Thread:** one of the processor's lanes of work. A different thread count can change tiny rounding and break exact regrowth.
- **zip / xz:** standard compression tools, used for comparison.
- **Drift / drift guard:** sliding from the goal toward the measuring tool; the guard is the written check against it.

13. Things outside the AI's control

- **The repository is public** on GitHub, with an MIT license: anyone can read and reuse everything in it, including this record. Some zine-project files there contain the owner's contact details. Changing visibility is the owner's decision, in GitHub settings. If a patent is planned, ask a patent attorney before more is made public.
- **The cloud machine is temporary.** Anything not committed and pushed is lost when it is deleted. The Aesop models (235 MB) live only on this machine.

- **Files cannot be placed on the owner's devices.** A cloud session cannot put files on a laptop desktop or phone home screen, or make widgets. Files are sent in the app and by email for the owner to save.
- **Scale honesty.** Every result comes from tiny models trained on one book. None of it changes how a large AI such as Claude understands anyone. Nothing has been tested on a large AI or in a real conversation; that needs an API key the owner would add.
- **Usage limits and safety checks.** Session limits stopped work twice on 20 Oct and twice on 21 Oct. On 21 Oct between 01:42 and 01:53 UTC, an automatic safety check blocked 5 replies to the owner's review request, with no reason shown. The AI cannot see or change why, and it has no access to billing or usage refunds (those go to Anthropic support).
- **Privacy in this record.** Some of the owner's messages and uploaded documents contain personal details and other people's names. Because this record is public, those are summarized in one neutral line or left out, never quoted. The owner still has the originals.